

Sumário

3. Fortran	3
3.1 INTRODUÇÃO	3
FORTTRAN: 1957	8
FORTTRAN II: 1958	8
FORTTRAN III: 1958	8
FORTTRAN IV: 1962	8
FORTTRAN 66: março de 1966	8
FORTTRAN 77: abril de 1978	9
Fortran 90: 1992	9
Fortran 95	10
Fortran 2003: 2004	10
Fortran 2008	10
3.2 BÁSICOS	16
bem_vindo.f90	16
quadrado.f90	20
aritmetica_basica.f90	22
intrinsicas.f90	27
Exercícios 3.2	29
3.3 COMANDOS DE CONTROLE E ARRAYS	33
lista_quadrados.f90	33
lista_quadrados2.f90	38
lista_quadrados3.f90	40
lista_quadrados4.f90	43
loteria.f90	46
lista_quadrados_dinamico.f90	50
lista_quadrados_dinamico_verifica.f90	52
matriz_produto.f90	55
matriz_produto_dinamica.f90	58
Exercícios 3.3	60
3.4 ACESSO A ARQUIVOS	61
loteria_sorteie_e lembre.f90	61

loteria_frequencia_sorteio.f90	70
Exercícios 3.4	75
3.5 FUNÇÕES E SUB-ROTINAS	76
area_circulo.f90	76
quadrado_e_cubo.f90	79
ordenar.f90	81
operacoes_circulo.f90	84
usa_operacoes_circulo.f90	84
argumentos_errados_MODULE.f90	87
argumentos_errados_principal.f90	87
Exercícios 3.5	90
3.6. OUTROS TIPOS DE DADOS	91
ordenar_nomes.f90	91
abrir_arquivo.f90	98
resposta.f90	100
resposta_case.f90	104
aritmetica_dupla.f90	106
aritmetica_complexa.f90	111
Exercícios 3.6	112
3.7. MAIS SOBRE ENTRADA E SAÍDA	113
abrir_arquivo_antigo_verificar.f90	113
abrir_arquivo_novo_verificar.f90	113
program loteria_sorteie_e lembre2	117
lista_quadrados_formatado.f90	122
aritmetica_dupla_formatado.f90	125
imprimir_numeros_grandes.f90	128
Arquivos internos	132
erro_read.f90	135
Exercícios 3.7	137

3. Fortran

- Uma linguagem de programação é uma linguagem artificial que pode ser usada para controlar o comportamento de uma máquina, particularmente um computador. As linguagens de programação, como línguas humanas, são definidas por regras sintáticas e semânticas, para determinar a estrutura e a significação respectivamente.

3.1 INTRODUÇÃO

A Linguagem de Programação da Ciência

- O **Fortran** (anteriormente **FORTRAN**) é uma linguagem de programação de uso geral, procedural, imperativa que é especialmente adequada à computação numérica e científica.
- O Fortran (um abreviatura de *The IBM Mathematical **For**mula **Trans**lating System*) abrange uma linhagem de versões, cada das quais evoluiu para acrescentar extensões à linguagem mantendo compatibilidade com versões anteriores.
- Os físicos geralmente não são “profissionais da computação”.
- O Fortran é a linguagem em que a maioria dos cálculos científicos e de engenharia é feito. Nas ciências físicas, foi estimado que 90% de todo tempo de computador é gasto executando programas que foram escritos em Fortran.
- O Fortran é a linguagem da ciência.
- O Fortran 90 é uma linguagem moderna.
- O Fortran 90 evolui continuamente.

funcionalidade	F77	C	C++	F90
robustez numérica	2	4	3	1
paralelismo de dados	3	3	3	1
abstração de dados	4	3	2	1
programação orientada para o objeto	4	3	1	2
programação funcional	4	3	2	1
média	3,4	3,2	2,2	1,2

Apropriado para cálculos numéricos

- Código rápido
- Tipos de dados de array úteis
- Clareza do código
- Portabilidade do código
- Disponibilidade de bibliotecas de sub-rotinas

Não recomendado para

- Programação de sistema operacional (use C!)
- Desenvolvimento de interfaces de usuário
- Aplicações de Internet
- Aplicações de bancos de dados



- O primeiro manual de Fortran apareceu em outubro de 1956, com o primeiro compilador Fortran sendo entregue em abril de 1957.
- Com o aumento da sua popularidade, em 1963 havia mais de 40 compiladores Fortran.



FORTRAN: 1957

FORTRAN II: 1958

A principal melhoria foi torná-la uma linguagem procedural permitindo sub-rotinas escritas pelo usuário e funções. Nos anos seguintes, FORTRAN II também acrescentaria o suporte de dados **DOUBLE PRECISION** e **COMPLEX**.

FORTRAN III: 1958

A IBM também desenvolveu um FORTRAN III em 1958 que permitia código de assembler inline entre outras características; contudo, esta versão nunca foi lançada como um produto.

FORTRAN IV: 1962

Por exigências dos clientes, a IBM desenvolveu o FORTRAN IV removendo as características dependentes da máquina do FORTRAN II, acrescentando novas características como o tipo de dados **LOGIC**, expressões lógicas Booleanas e o **IF** lógico como uma alternativa ao **IF** aritmético.

FORTRAN 66: março de 1966

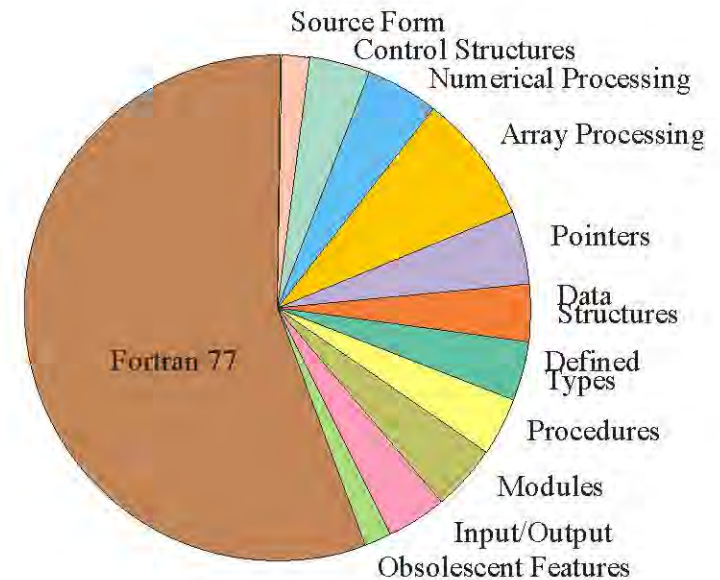
O primeiro padrão ANSI (American National Standards Institute) foi lançado que definiu uma base sólida para subsequentes desenvolvimentos da linguagem. Foi o primeiro padrão para uma linguagem de programação.

FORTRAN 77: abril de 1978

O segundo padrão ANSI foi lançado que padronizou extensões, permitindo a programação estruturada, e introduziu novas características para o IF e o tipo de dados de CHARACTER.

Fortran 90: 1992

Depois de muitos atrasos, o sucessor do FORTRAN 77, informalmente conhecido como Fortran 90, foi finalmente lançado como padrão ISO em 1991 e o terceiro padrão ANSI em 1992. Esta grande revisão acrescentou muitas novas características para refletir as mudanças significativas na prática de programação que se tinha desenvolvido desde o padrão 1978.



Fortran 95

Uma pequena revisão, principalmente para resolver algumas questões do padrão Fortran 90. Também acrescentou um número de extensões, notavelmente a especificação do Fortran de Alto Desempenho.

Fortran 2003: 2004

Publicado em 18 de novembro de 2004, este é o padrão mais recente. Ele é uma revisão grande que introduz muitas novas características.

Fortran 2008

Esforços estão sendo feitos para desenvolver uma revisão do Fortran 2003, tentativamente chamado Fortran 2008. Como com o Fortran 95, a intenção é que seja uma pequena

atualização, incorporando clarificações e correções ao Fortran 2003, bem como introduzindo umas poucas novas capacidades.

Conjunto de caracteres do Fortran

O alfabeto especial usado na linguagem Fortran 90 é conhecido como o *conjunto de caracteres do Fortran*. É composto dos 86 símbolos mostrados na Tabela abaixo.

Número de Símbolos	Tipo	Valores
26	Letras maiúsculas	A–Z
26	Letras minúsculas	a–z
10	Dígitos	0–9
1	Sublinhado	_
5	Símbolos aritméticos	+ - * / **
17	Outros símbolos	() . = , ' \$: ! " % & ; < > ? e espaço

Conteúdo deste capítulo

- O Fortran é uma linguagem extensa e este curso introdutório só cobrirá um subconjunto dela.
- O método adotado neste curso é aprender pelo exemplo e introduzir novas características de linguagem à medida que forem necessários para realizar tarefas adicionais dentro desses exemplos.
- A avaliação é através dos programas escritos por vocês. Para aprender tem que “quebrar a cabeça”.

Linux básico

- O Linux tem uma interface gráfica poderosa e pode-se fazer muito através dela. Há dois tipos disponíveis: Gnome e KDE. Você tem que escolher qual quer usar antes de fazer o login. Explore os menus e vai descobrir muita coisa. Clicando duas vezes no ícone com seu nome no desktop vai abrir uma janela com o conteúdo do seu diretório.
- Para compilar e executar os programas você vai precisar teclar comandos em uma janela de terminal. No Gnome, esta janela pode ser aberta no menu Applications > System Tools > Terminal.
- Na janela de terminal, o usuário interage com Linux através de um tipo de programa chamado Shell. Há vários, mas o nosso padrão aqui é usar o bash. O Linux é distingue entre letras maiúsculas e minúsculas, assim o comando `ls` é diferente do `LS` ou do `Ls`.

Comandos básicos:

<code>ls</code>	Mostra os arquivos do diretório de trabalho corrente.
<code>pwd</code>	Mostra o nome do arquivo de trabalho atual
<code>cd <i>diretório</i></code>	Muda de diretório
<code>cat <i>arquivo_texto</i></code>	Mostra o conteúdo <i>arquivo_texto</i> na tela
<code>less <i>arquivo_texto</i></code>	Mostra o conteúdo <i>arquivo_texto</i> no monitor uma tela de cada vez.
<code>cp <i>arquivo1 arquivo2</i></code>	Copia um arquivo ou diretório
<code>mv <i>arquivo1 arquivo2</i></code>	Move um arquivo ou diretório
<code>mkdir <i>diretório</i></code>	Cria <i>diretório</i>
<code>touch <i>arquivo</i></code>	Cria <i>arquivo</i>

```
[exemplo@lafcl ~]$ ls
Navio_Negreiro
[exemplo@lafcl ~]$ ls -l
total 8
-rw-r--r-- 1 exemplo users 8163 2009-04-07 17:08 Navio_Negreiro
[exemplo@lafcl ~]$ ls -a
. .bash_history .bash_profile .emacs .kde Navio_Negreiro .zshrc
.. .bash_logout .bashrc .gnome2 .mozilla .viminfo
[exemplo@lafcl ~]$ ls -la
total 56
drwx----- 5 exemplo users 4096 2009-04-07 17:08 .
drwxr-xr-x 50 root root 4096 2009-04-07 16:59 ..
-rw----- 1 exemplo users 5 2009-04-07 16:58 .bash_history
-rw-r--r-- 1 exemplo users 18 2008-10-23 11:30 .bash_logout
-rw-r--r-- 1 exemplo users 176 2008-10-23 11:30 .bash_profile
-rw-r--r-- 1 exemplo users 124 2008-10-23 11:30 .bashrc
-rw-r--r-- 1 exemplo users 500 2008-05-23 14:35 .emacs
drwxr-xr-x 2 exemplo users 4096 2008-04-06 17:43 .gnome2
drwx----- 3 exemplo users 4096 2009-04-07 16:58 .kde
drwxr-xr-x 4 exemplo users 4096 2008-10-29 06:18 .mozilla
-rw-r--r-- 1 exemplo users 8163 2009-04-07 17:08 Navio_Negreiro
-rw----- 1 exemplo users 872 2009-04-07 17:08 .viminfo
-rw-r--r-- 1 exemplo users 658 2008-05-16 01:17 .zshrc
[exemplo@lafcl ~]$ pwd
/home/lafc/exemplo
[exemplo@lafcl ~]$ cd
[exemplo@lafcl ~]$ cd ..
[exemplo@lafcl lafc]$ pwd
/home/lafc
[exemplo@lafcl lafc]$ cd
[exemplo@lafcl ~]$ pwd
/home/lafc/exemplo
[exemplo@lafcl ~]$ cat Navio_Negreiro
Navio Negreiro
```

Castro Alves

I

'Stamos em pleno mar... Doudo no espaço
Brinca o luar – dourada borboleta;
E as vagas após ele correm... cansam
Como turba de infantes inquieta.

```
.
. Vai ser mostrado todo o arquivo na tela
.
```

Fatalidade atroz que a mente esmaga!

Extingue nesta hora o brigue imundo
O trilho que Colombo abriu nas vagas,
Como um íris no pélagos profundo!
Mas é infâmia demais! ... Da etérea plaga
Levantai-vos, heróis do Novo Mundo!
Andrada! arranca esse pendão dos ares!
Colombo! fecha a porta dos teus mares!

```
[exemplo@lafcl ~]$ less Navio_Negreiro
Navio Negreiro
Castro Alves
```

I

'Stamos em pleno mar... Doudo no espaço
Brinca o luar – dourada borboleta;
E as vagas após ele correm... cansam
Como turba de infantes inquieta.

'Stamos em pleno mar... Do firmamento
Os astros saltam como espumas de ouro...
O mar em troca acende as ardências,
– Constelações do líquido tesouro...

'Stamos em pleno mar... Dois infinitos
Ali se estreitam num abraço insano,
Azuis, dourados, plácidos, sublimes...
Qual dos dous é o céu? qual o oceano?...

'Stamos em pleno mar. . . Abrindo as velas
Navio_Negreiro

```
[exemplo@lafcl ~]$ cp Navio_Negreiro NN
[exemplo@lafcl ~]$ ls
Navio_Negreiro NN
[exemplo@lafcl ~]$ ls -l
total 16
-rw-r--r-- 1 exemplo users 8163 2009-04-07 17:08 Navio_Negreiro
-rw-r--r-- 1 exemplo users 8163 2009-04-07 17:12 NN
[exemplo@lafcl ~]$ mv NN NN1
[exemplo@lafcl ~]$ ls -l
total 16
-rw-r--r-- 1 exemplo users 8163 2009-04-07 17:08 Navio_Negreiro
-rw-r--r-- 1 exemplo users 8163 2009-04-07 17:12 NN1
[exemplo@lafcl ~]$ rm NN1
[exemplo@lafcl ~]$ ls -l
total 8
-rw-r--r-- 1 exemplo users 8163 2009-04-07 17:08 Navio_Negreiro
[exemplo@lafcl ~]$ mkdir Castro_Alves
[exemplo@lafcl ~]$ ls -l
```

```
total 12
drwxr-xr-x 2 exemplo users 4096 2009-04-07 17:13 Castro_Alves
-rw-r--r-- 1 exemplo users 8163 2009-04-07 17:08 Navio_Negreiro
[exemplo@lafcl ~]$ ls
Castro_Alves  Navio_Negreiro
[exemplo@lafcl ~]$ mv Navio_Negreiro Castro_Alves/
[exemplo@lafcl ~]$ ls -l
total 4
drwxr-xr-x 2 exemplo users 4096 2009-04-07 17:13 Castro_Alves
[exemplo@lafcl ~]$ cd Castro_Alves/
[exemplo@lafcl Castro_Alves]$ ls -l
total 8
-rw-r--r-- 1 exemplo users 8163 2009-04-07 17:08 Navio_Negreiro
[exemplo@lafcl Castro_Alves]$ touch Espumas_Flutuantes
[exemplo@lafcl Castro_Alves]$ ls -l
total 8
-rw-r--r-- 1 exemplo users 0 2009-04-07 17:15 Espumas_Flutuantes
-rw-r--r-- 1 exemplo users 8163 2009-04-07 17:08 Navio_Negreiro
[exemplo@lafcl Castro_Alves]$
```

3.2 BÁSICOS

bem_vindo.f90

Este exemplo introduz as características de linguagem:

- Comandos
- Compilação e execução
- Comentários
- Print
- End

Não se usa acentos ou caracteres especiais. O Fortran não distingue letras maiúsculas de minúsculas. Os programas de Fortran são normalmente armazenados em arquivos com uma extensão `.f90`.

```
PROGRAM bem_vindo
! Programa para escrever uma mensagem de boas-vindas

    IMPLICIT none

    PRINT *, "Bem-vindo ao Curso Introdutorio de Fortran 90!"

END PROGRAM bem_vindo
```

Com os espaços visíveis (aqui • representa um espaço em branco)

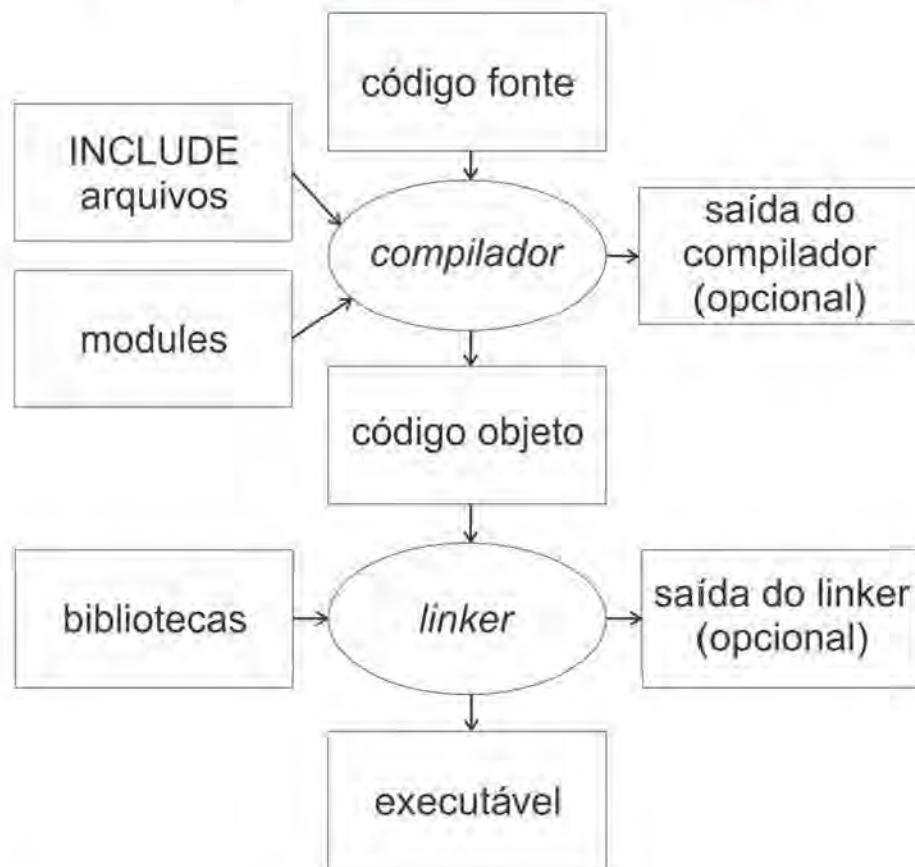
```
PROGRAM•bem_vindo
!•Programa•para•escrever•uma•mensagem•de•boas-vindas

...IMPLICIT•none

...PRINT•*, "Bem-vindo•ao•Curso•Introdutorio•de•Fortran•90!"

END•PROGRAM•bem_vindo
```

Compilação e linkagem



Compilação e execução do programa

Para compilar o programa `bem_vindo.f90` em um computador usando Linux o comando é

```
[heber@lafc ~]$ gfortran -o bem_vindo bem_vindo.f90
[heber@lafc ~]$ ./bem_vindo
Bem-vindo ao Curso Introdutorio de Fortran 90!
```

Notas de layout

- Linhas e espaços em branco.
- As linhas podem ter até 132 caracteres. Entretanto, use até 70 colunas para ser mais legível.
- & é o caráter de continuação.

```
PROGRAM bem_vindo
! Programa para escrever
!      uma mensagem de boas-vindas
IMPLICIT none
PRINT *,&
      "Bem-vindo ao Curso Introdutorio de Fortran 90!"
END PROGRAM bem_vindo
```

OU

```
PROGRAM bem_vindo
! Programa para escrever
!      uma mensagem de boas-vindas
IMPLICIT none
PRINT *, "Bem-vindo ao Curso Introdutorio de &
      &Fortran 90!"
END PROGRAM bem_vindo
```

Constantes literais caráter

Válidas:

```
'Esta eh uma constante de 37 caracteres'
"      "
'45.68'
```

Inválidas:

```
Constante carater invalida
'Outra
```

Para incluir um apóstrofo ou aspas dentro de uma constante caráter, eles devem aparecer duplicados, por exemplo,

```
"P'ra que ""isto""?" = 'P'ra que "isto"?'"
```

De qualquer forma será impresso:

```
P'ra que "isto"?
```

Este código Fortran está escrito corretamente?

```
PRINT *, 'Acordado de manha,&  
dormindo aa noite.'
```

A palavra da língua inglesa default é muito usada. Ela significa a maneira como algo vai aparecer ou ser feito se você não fizer outra escolha. É o padrão.

quadrado.f90

Este exemplo introduz novas características da linguagem

- **Variáveis**
- **Tipo INTEGER**
- **READ**
- **Atribuição**

```
PROGRAM quadrado  
! Programa para calcular quadrados  
  
! Declaracoes  
  IMPLICIT none  
  INTEGER :: numero, numero_quadrado  
  
! Comandos executaveis  
  PRINT *, "Bem-vindo ao programa quadrado."  
  PRINT *  
  PRINT *, "Digite um numero inteiro e &  
                                     &tecle Enter:"  
  
  READ *, numero  
  numero_quadrado = numero * numero  
  PRINT *, numero, " ao quadrado eh ",&  
                                     numero_quadrado  
  
END PROGRAM quadrado
```

```
[heber@lafc ~]$ gfortran -o quadrado quadrado.f90
```

```
[heber@lafc ~]$ ./quadrado
```

```
Bem-vindo ao programa quadrado
```

```
  Digite um numero inteiro e tecla Enter
```

```
12
```

```
12  ao quadrado eh
```

```
144
```

Nomes

Podem ter de 1 a 31 caracteres alfanuméricos (letras, underscores=traço inferior ou sublinhado) e numerais dos quais o primeiro deve ser uma letra.

Variável definida e indefinida

Antes de ser usada, uma variável precisa ser inicializada (ter um valor armazenado nela), caso contrário seu conteúdo estará indefinido.

Constantes literais inteiras:

Válidas

-3478
0
567890
+12345678

Inválidas

-1,000
987.
987654321098

Tipicamente o tipo de dados **inteiro** tem valores restritos ao intervalo -2147483648 a +2147483647. Se uma operação matemática resultar em um valor fora deste intervalo o resultado será inválido (overflow).

Vários comandos curtos podem ser escritos em uma mesma linha:

A = 0; B = 0; C = 0

aritmetica_basica.f90

- Operadores aritméticos
- Tipo REAL

```
PROGRAM aritmetica_basica
! Exemplos de aritmetica basica

IMPLICIT none
INTEGER :: primeiraparte, segundaparte, &
           soma, diferenca
INTEGER :: i_numerador, i_denominador, &
           i_quociente, numero
REAL :: largura, comprimento, area, &
        numerador, denominador, quociente

! Adicao
primeiraparte = 23
segundaparte = 19
soma = primeiraparte + segundaparte
PRINT *, "A soma de ", primeiraparte, &
        " e ", segundaparte, " eh ", soma

! Subtracao
diferenca = primeiraparte - segundaparte
PRINT *
PRINT *, primeiraparte, " menos ", &
        segundaparte, " eh ", diferenca

! Multiplicacao
largura = 2.3
comprimento = 5.47
area = largura * comprimento
PRINT *
PRINT *, "A area do retangulo com largura ", &
        largura, " e comprimento ", &
```

```

                                comprimento, " eh ", area
! Divisao
numerador = 22.0
denominador = 7.0
quociente = numerador / denominador
PRINT *
PRINT *, "A divisao de ", numerador, &
        " por ", denominador, " da ", quociente

! Divisao INTEGER
i_numerador = 5
i_denominador = 3
i_quociente = i_numerador / i_denominador
PRINT *
PRINT *, "A divisao inteira de ", &
        i_numerador, " por ", i_denominador, &
        " da ", i_quociente

! Exponenciacao
numero = 7
PRINT *
PRINT *, numero, " ao cubo eh ", numero**3

! Cuidado
PRINT *
PRINT *, numero, " elevado a -3 eh ", &
        numero**(-3)

END PROGRAM aritmetica_basica

```

```

$ gfortran -o aritmetica_basica aritmetica_basica.f90
$ ./aritmetica_basica
A soma de          23 e          19 eh
42

          23 menos          19 eh          4

```

```

A area do retangulo com largura    2.300000    e
comprimento    5.470000 eh    12.58100

A divisao de    22.00000    por    7.000000    da
3.142857

A divisao inteira de                5 por                3
da                1

          7 ao cubo eh                343

          7 elevado a -3 eh                0

```

Tipicamente o tipo de dados **REAL** ocupam 32 bits e tem valores restritos ao intervalo $1.175494\text{E}-38$ a $3.402823\text{E}+38$ e têm uma precisão de seis algarismos significativos decimais. Se uma operação matemática resultar em um valor acima do valor máximo permitido ocorre um overflow e o resultado será inválido. Se uma operação matemática resultar em um valor abaixo do valor mínimo permitido ocorre um underflow e o resultado será zero.

Constantes literais reais:

Válidas

```

0.0123 = 1.23E-02
0.0 = 0.0E0
-23456.0 = -2.3456E4
+987652.0 = 9.87652E+05
2. = 2.0

```

Inválidas

```

-10
1,123
145678E4
666888.E8.2

```

A maioria dos operadores é binário: $a + b$, mas existe um operador unário: $-x$.

Prioridade:

$**$ $*$ / $+$ $-$

Precedência:

$-A+B+C = ((-A)+B)+C$

$A**B**C = A**(B**C)$

Exemplo:

$1.0 + 2.0*3.0 / (6.0*6.0 + 5.0*44.0)**0.25$

$1.0 + 2.0*3.0 / (36.0 + 5.0*44.0) ** 0.25$

$1.0 + 2.0*3.0 / (36.0 + 5.0*44.0) ** 0.25$

$1.0 + 2.0*3.0 / (36.0 + 220.0) ** 0.25$

$1.0 + 2.0*3.0 / 256.0 ** 0.25$

$1.0 + 2.0*3.0 / 4.0$

$1.0 + 6.0 / 4.0$

$1.0 + 1.5$

2.5

Exemplos de expressões inválidas

Incorreto

$A**-B$

$A(B+3.6)$

$A*2.75-B*(C+D)$

Correto

$A**(-B)$

$A*(B+3.6)$

$A*(2.75-B*(C+D))$

O que significa a seguinte linha de código?

$A = 0.0$; $B = 370$! Inicializacao ; $C = 17.0$; $D = 33.0$

Estas declarações estão corretas?

REAL :: real; REAL :: preco

REAL :: pi = 22/7

REAL :: x = 2., y = -3

Este código Fortran está escrito corretamente?

x = 0.4-6

limite-inferior = 0.0005E10

y = E6

intrinsecas.f90

- Funções intrínsecas
- Constantes nominadas

```
PROGRAM intrinsecas
! Exemplos de funcoes intrinsecas

IMPLICIT none
INTEGER :: i_grande, i_pequeno
REAL :: angulo_em_radianos, quadrado, &
        grande, pequeno

REAL, PARAMETER :: pi = 3.14159265

! sqrt() dah a raiz quadrada de um numero
quadrado = 64.0
PRINT *, "A raiz quadrada de ", quadrado, &
        " eh ", sqrt( quadrado )

! sin() dah o seno de um angulo em radianos
angulo_em_radianos = pi / 6.0
PRINT *
PRINT *, "O seno de pi/6 radianos e ", &
        sin( angulo_em_radianos )

! max() dah o valor maximo de
! uma lista de numeros
grande = 98.4
pequeno = 32.0
PRINT *
PRINT *, "O maior entre ", grande, " e ", &
        pequeno, " eh ", max( grande, pequeno )
```

```
! min() dah o valor minimo de uma
! lista de numeros

i_grande = 999
i_pequeno = 7
PRINT *
PRINT *, "O menor entre ", i_grande, " e ", &
        i_pequeno, " eh ", &
        min( i_grande, i_pequeno )

END PROGRAM intrinsecas
```

```
$ gfortran -o intrinsecas intrinsecas.f90
$ ./intrinsecas
A raiz quadrada de      64.00000      eh
8.000000

O seno de pi/6 radianos e      0.5000000

O maior entre      98.40000      e      32.00000
eh      98.40000

O menor entre      999 e      7 eh
7
```

Exemplos de Funções Intrínsecas

Funções Numéricas

ABS(A)	Valor absoluto
REAL(A)	Conversão para real
INT(A)	Conversão para inteiro (trunca) INT(1.2345) → 1, INT(-2.987) → -2
NINT(A)	Conversão para inteiro (mais próximo) NINT(0.5) → 1, NINT(-0.5) → -1
FLOOR(A)	Converte para o maior inteiro menor ou igual a A FLOOR(2.345) → 2, FLOOR(-2.345) → -3
CEILING(A)	Converte para o menor inteiro maior ou igual a A CEILING(2.345) → 3, CEILING(-2.345) → -2

Funções Matemáticas

ACOS(X)	arccos	EXP(X)	
ASIN(X)	arcsin	LOG(X)	ln
ATAN(X)	arctan	SIN(X)	
COS(X)		SINH(X)	
COSH(X)		TAN(X)	
		TANH(X)	

- Funções têm a maior prioridade.
- Argumentos de funções são calculados primeiro.
- O valor da função retornado é tratado como um valor na expressão.

Exemplo:

REAL :: A = 1.0, B = -5.0, C = 6.0, R

R = (-B + SQRT(B*B - 4.0*A*C)) / (2.0*A)

$$\frac{-B + \sqrt{B*B - 4.0*A*C}}{2.0*A}$$

--> $\frac{5.0 + \sqrt{B*B - 4.0*A*C}}{2.0*A}$
--> $\frac{5.0 + \sqrt{25.0 - 4.0*A*C}}{2.0*A}$
--> $\frac{5.0 + \sqrt{25.0 - 4.0*C}}{2.0*A}$
--> $\frac{5.0 + \sqrt{25.0 - 24.0}}{2.0*A}$
--> $\frac{5.0 + \sqrt{1.0}}{2.0*A}$
--> $\frac{5.0 + 1.0}{2.0*A}$
--> $\frac{6.0}{2.0*A}$
--> $\frac{6.0}{2.0}$
--> 3.0 ! Que eh o valor de R

Exercícios 3.2

Exercício 3.2.1

Modifique o programa `quadrado.f90` para imprimir o cubo e a quarta potência do número ao invés do quadrado. Você pode identificar potenciais limitações deste programa?

Exercício 3.2.2

Escreva um programa para pedir ao usuário o mês (1 a 12) e ano do seu nascimento e imprimir a sua idade (em anos). Use apenas aritmética, constantes e variáveis inteiras.

Exercício 3.2.3

Escreva um programa para pedir o comprimento e a largura de uma sala em metros. Considerando que o carpete é vendido com 3m de largura, calcule o menor comprimento (em metros inteiros) necessário para cobri-la. Inclua os comentários necessários para justificar geometricamente a forma como os cálculos foram feitos. (Dica: as funções intrínsecas `ceiling` e `min` podem ajudar). Reproduza os resultados abaixo

Largura	Compr.	Carpete
13.4	11.7	54
11.7	13.4	54
30.1	3.2	36
3.2	30.1	36
19.8	2.7	19
2.7	19.8	19
29.3	30.5	305
30.5	29.3	305
12.8	29.5	128
29.5	12.8	128
12.0	11.0	44
11.0	12.0	44

- Enviem o programa fonte (arquivo.f90) e o teste (arquivo.txt) como anexos de um email enviado para `IFC.2015.1@fis.ufal.br`. O assunto do email deve ser o número do exercício, por exemplo, **Exercício 3.2.2**. O teste é a prova de que o programa compila e executa corretamente.
- Se o programa não estiver correto, responderei o email informando o(s) problema(s). Se eu fizer uma pergunta, responda-a. Responda o meu email com o programa corrigido e seu teste. O texto do email de resposta deve listar as correções solicitadas e as medidas tomadas. A nova versão do programa deve ser executada e testada novamente.
- O exercício só será aceito quando estiver correto (Programa OK).
- Procurem entregar o exercício no prazo especificado. Programas enviados após o prazo serão aceitos, mas a nota cai com o atraso.

3.3 COMANDOS DE CONTROLE E ARRAYS

lista_quadrados.f90

Este exemplo introduz as características da linguagem

- **Arrays**
- **DO loops**

```
PROGRAM lista_quadrados
! Exemplos de arrays e lacos DO

IMPLICIT none
INTEGER :: numero
INTEGER, PARAMETER :: tamanho = 20
INTEGER :: quadrados(tamanho)

! Armazena os primeiros "tamanho" quadrados
DO numero = 1, tamanho
    quadrados( numero ) = numero ** 2
END DO

! Imprima a lista de quadrados
PRINT *, "Os primeiros 20 quadrados &
        &sao: ", quadrados

END PROGRAM lista_quadrados
```

```
$ gfortran -o lista_quadrados \
> lista_quadrados.f90
$ ./lista_quadrados
Os primeiros 20 quadrados sao: 1
4          9          16          25
36         49         64         81
100        121        144        169
196        225        256        289
324        361        400
```

A forma geral do comando **DO limitado** (DO com um contador de iteração) é

```
[nome:] DO variável = expr1, expr2[, expr3]
        bloco
END DO [nome]
```

onde *variável* é uma variável inteira escalar. Qualquer comando Fortran que seja parte de um comando composto pode ser rotulado para identificá-lo.

Advertências!

- Nunca use qualquer uma das instruções dentro do loop para alterar as variáveis do comando DO.
- Não suponha que o índice do DO terá o valor *expr2* após a conclusão normal do loop DO. Suponha que ele é indefinido. Entretanto, se ocorrer a saída do laço DO antes da conclusão normal, por exemplo, pela utilização de uma instrução EXIT (ver adiante), então o índice reterá o seu valor.

Este tipo de DO é útil para repetir um conjunto de comandos FORTRAN um número **fixo** de vezes.

Quantas vezes os seguintes loops serão executados, quais os valores da variável *i* do laço dentro do loop?

```
DO i = 1, 5
DO i = 5, 0, -1
DO i = 10, 1, -2
DO i = 0, 30, 7
DO i = 3, 2, 1
```

Arrays de tipo intrínseco

Um array consiste de um conjunto retangular de elementos, todos do mesmo tipo. Pode ser declarado assim

```
REAL, DIMENSION(10) :: a
```

ou assim

```
REAL :: a(10)
```

Os elementos sucessivos deste array são *a*(1), *a*(2), *a*(3), ..., *a*(10). O número de elementos de um array é chamando *tamanho*.

Podemos também declarar o *limite* inferior de um array.

```
REAL, DIMENSION(-10:5) :: vetor
```

ou

```
REAL :: vetor(-10:5)
```

O tamanho deste array é 16, *vetor*(-10), *vetor*(-9), ..., *vetor*(5).

Um array pode ter mais de uma dimensão e o Fortran permite até sete.

Como fizemos acima, se refere a um elemento individual de um array usando valores do *subscrito*, que pode ser qualquer *expressão inteira escalar*, por exemplo

```
a(1)
a(i*j) ! i e j sao do tipo INTEGER
a(NINT(x+3.)) ! x eh do tipo REAL.
```

Também se pode referir a subarrays, chamados *seções*, especificando um intervalo de subscritos:

```
a(i:j) ! array de tamanho j-i+1
y(1:10) = x(11:20)
```

No caso geral, a seção é especificada por um trio de números da forma:

```
[índice] : [índice] [: passo]
```

Exemplos,

```
ARRAY_A (1:N:2,M) ! unidimensional
B (1:4:3, 6:8:2, 3) ! bidimensional
```

O segundo membro é calculado primeiro

```
x(1:10) = x(10:1:-1)
```

Pode-se usar um array inteiro como um objeto, por exemplo,

```
y = 2.0*cos(x) + 1.
```

onde *x* e *y* são arrays do tipo REAL e de limites iguais.

Funções intrínsecas para arrays

Função	Exemplo	Efeito
product	product(a)	Produto dos elementos de um array <i>a</i>
sum	sum(a)	Soma dos elementos de um array <i>a</i>
dot_product	dot_product(v1,v2)	Produto interno dos vetores <i>v1</i> e <i>v2</i>
matmul	matmul(a,b)	Multiplica duas matrizes <i>a</i> e <i>b</i>
maxval	maxval(a)	Valor máximo no array <i>a</i>
minval	minval(a)	Valor mínimo no array <i>a</i>

Uma constante do tipo array tem a forma

```
(/ 1, 2, 3, 5, 10 /)  
(/ 1, 2, 3, 4, 5 /) ou (/ (I, I= 1,5) /)  
(/ 2, 4, 6, 8 /) ou (/ (I, I= 2,8,2) /)  
(/ 1.1,1.2,1.3 /) ou (/ (I*0.1, I= 11,13)/)
```

Regras para um alinhamento correto:

1. Escolha um deslocamento: geralmente 3 ou 4 colunas.
 2. Os comandos dentro de um nível devem estar alinhados.
 3. A cada nível os comandos são deslocados para a direita da mesma quantidade de colunas.
-

lista_quadrados2.f90

Este exemplo introduz as características da linguagem

- Limite variável no *DO* loop
- Lista com *DO*-implícito na impressão

```
PROGRAM lista_quadrados2  
! Exemplos de arrays e lacos do  
! usando limite variavel do limite do loop  
! e lista com Do-implicito na impressao.  
  
    IMPLICIT none  
    INTEGER, PARAMETER :: tamanho = 20  
    INTEGER :: quadrados(tamanho)  
    INTEGER :: numero, quantidade_numeros  
  
! Find out how many quadrados are required  
    PRINT *, "Quantos quadrados voce quer?"  
    READ *, quantidade_numeros  
  
! Armazena os quadrados  
    DO numero = 1, quantidade_numeros  
        quadrados( numero ) = numero ** 2  
    END DO  
  
! Imprima a lista de quadrados  
    PRINT *  
    PRINT *, "Os primeiros ", &  
        quantidade_numeros, &  
        " quadrados sao: ", &  
        ( quadrados(numero), numero = &  
            1, quantidade_numeros )  
! ou  
    quadrados(1:quantidade_numeros)  
END PROGRAM lista_quadrados2
```

```

$ gfortran -o lista_quadrados2 \
> lista_quadrados2.f90
$ ./lista_quadrados2
  Quantos quadrados voce quer?
10

  Os primeiros          10  quadrados sao:
1           4           9           16
25          36          49          64
81          100

$ lista_quadrados2
  Quantos quadrados voce quer?
40
Segmentation fault

$ gfortran -fbounds-check -o \
> lista_quadrados2 lista_quadrados2.f90
$ ./lista_quadrados2
  Quantos quadrados voce quer?
40
Fortran runtime error: Array reference out
of bounds for array 'quadrados', upper bound
of dimension 1 exceeded (in file
'lista_quadrados2.f90', at line 16)

```

lista_quadrados3.f90

Este exemplo introduz as características da linguagem

- **Bloco ou construto IF**

```

PROGRAM lista_quadrados3
! Exemplo de arrays e DO loops introduzindo
! o bloco if

IMPLICIT none
INTEGER, PARAMETER :: &
                                maximo_permitido = 20
INTEGER :: quadrados( maximo_permitido )
INTEGER :: numero, quantidade_numeros

! Ache quantos quadrados sao requeridos
PRINT *, "Quantos quadrados voce quer?"
READ *, quantidade_numeros

! Verifique se esta dentro do limite
!
IF( quantidade_numeros > &
                                maximo_permitido ) THEN
PRINT *
PRINT *, "Isto eh demais. &
                                &O maximo permitido eh ", &
                                maximo_permitido
ELSE
! Quantidade_numeros esta OK entao
! armazene os quadrados
DO numero = 1, quantidade_numeros
quadrados( numero ) = numero ** 2
END DO

```

```

!      Imprima a lista dos quadrados
      PRINT *
      PRINT *, "Os primeiros ", &
             quantidade_numeros, &
             " quadrados sao: ", &
             ( quadrados(numero), numero &
               = 1, quantidade_numeros )
! ou      quadrados(1:quantidade_numeros)
      END IF
END PROGRAM lista_quadrados3

```

```

$ gfortran -o lista_quadrados3 \
> lista_quadrados3.f90
$ ./lista_quadrados3
  Quantos quadrados voce quer?
99

  Isto eh demais.  O maximo permitido eh
20
$ ./lista_quadrados3
  Quantos quadrados voce quer?
3

  Os primeiros          3  quadrados sao:
1              4          9

```

Forma mais simples da construção *IF*
 [nome:] IF (*expressão-lógica-escalar*) THEN
 bloco

END IF [nome]

(nada impede que o *bloco* contenha *IFs*).

Segunda forma da construção *IF*

[nome:] IF (*expressão-escalar-lógica*) THEN
 bloco1
 ELSE [nome]
 bloco2
 END IF [nome]

Operadores relacionais escalares

.LT.	ou <	menor do que
.LE.	ou <=	menor ou igual
.EQ.	ou ==	igual
.NE.	ou /=	diferente
.GT.	ou >	maior do que
.GE.	ou >=	maior ou igual

Variáveis podem ser inicializadas em sua definição:

INTEGER :: n = 20

lista_quadrados4.f90

Este exemplo introduz a característica da linguagem

- Ninho de IFs

```
PROGRAM lista_quadrados4
! Exemplo de arrays e DO loops
! mostrando ninho de de IFs

IMPLICIT none
INTEGER, PARAMETER :: &
    maximo_permitido = 20
INTEGER :: quadrados(maximo_permitido)
INTEGER :: numero, quantidade_numeros

! Ache quantos quadrados sao necessarios
PRINT *, "Quantos quadrados voce quer?"
READ *, quantidade_numeros

! Verifique se estah dentro do limite
! permitido
max: IF( quantidade_numeros > &
    maximo_permitido ) THEN
    PRINT *
    PRINT *, "Isto eh demais. &
        &O maximo permitido eh ", &
            maximo_permitido
ELSE max
min: IF( quantidade_numeros < 1 ) THEN
    PRINT *
    PRINT *, quantidade_numeros, &
        " nao eh um numero valido!"
ELSE min
```

```
! Quantidade_numeros esta OK entao
! armazene os quadrados
DO numero = 1, quantidade_numeros
    quadrados( numero ) = &
        numero ** 2
END DO

! Imprimir a lista dos quadrados
PRINT *
so_um: IF( quantidade_numeros = 1 ) &
    THEN
    PRINT *, &
        "O primeiro quadrado eh: "&
            , quadrados( 1 )
ELSE so_um
    PRINT *, "Os primeiros ", &
        quantidade_numeros, &
        " quadrados sao: ", &
        ( quadrados(numero), &
            numero =&
                1, quantidade_numeros )
END IF so_um
END IF min
END IF max
END PROGRAM lista_quadrados4
```

```

$ gfortran -o lista_quadrados4 \
> lista_quadrados4.f90
$ ./lista_quadrados4
  Quantos quadrados voce quer?
1
  O primeiro quadrado eh:          1
lista_quadrados4
  Quantos quadrados voce quer?
2

  Os primeiros          2  quadrados sao:
1              4

```

Regras para um alinhamento correto:

1. Escolha um deslocamento: geralmente 3 ou 4 colunas.
2. Os comandos dentro de um nível devem estar alinhados.
3. A cada nível os comandos são deslocados para a direita na mesma quantidade de colunas que o deslocamento inicial.
4. O comando e não seu nome deve estar alinhado.

loteria.f90

Este exemplo introduz as características da linguagem

- Tipo **LOGICAL**
- **DO** loop ilimitado
- Comando **EXIT**
- Comando **IF**
- Sub-rotinas Intrínsecas

```

PROGRAM loteria
! Programa de sorteio de loteria
!     introduzindo o tipo LOGICAL,
!     loops indefinidos, comando EXIT
!     e sub-rotinas intrinsecas

    IMPLICIT none
    INTEGER, PARAMETER :: total_numeros = 6
    INTEGER :: sorteio_numeros(total_numeros)
    INTEGER :: numeros_sorteados = 0
    INTEGER :: proxima_escolha, isorteio
    REAL :: xaleatorio
    LOGICAL :: diferente

! Inicialize o gerador de numeros aleatorios
!     (Dependente do sistema)
    CALL random_seed

! Repita ate termos total_numeros
!     numeros distintos
    DO
        IF( numeros_sorteados == &
            total_numeros ) EXIT
        ! xaleatorio eh um numero aleatorio
        !     no intervalo [0.0, 1.0)
        CALL random_number( xaleatorio )
    
```

```

! Multiplique-o por 49 para obter um
!   numero aleatorio no intervalo
!   [0.0, 49.0).
   xaleatorio = xaleatorio * 49.0
! Arredonde para baixo e some 1
!   para obter um numero inteiro
!   entre 1 e 49.
   proxima_escolha = &
                        floor( xaleatorio ) + 1

! veja se eh diferente dos numeros
!   selecionados ate agora e se for
!   adicione-o a lista
   diferente = .true.
DO isorteio = 1, numeros_sorteados
   IF( proxima_escolha == &
       sorteio_numeros&
           ( isorteio ) ) THEN
       diferente = .false.
       EXIT
   END IF
END DO
IF( diferente ) THEN
   numeros_sorteados = &
                       numeros_sorteados + 1
   sorteio_numeros&
       ( numeros_sorteados ) =&
       proxima_escolha

   END IF
END DO

PRINT *, "Os numeros da loteria sao: ", &
       sorteio_numeros

END PROGRAM loteria

```

```

$ gfortran -o loteria loteria.f90
$ ./loteria
   Os numeros da loteria sao:           49
28                48                37                19                24

```

Uma variável lógica pode ter somente dois valores **.true.** ou **.false.**

A forma mais simples do *IF* é o comando *IF*
 [nome:] IF (expressão-lógica-escalar) comando

Uma forma muito simples do comando *DO* é a ilimitada (às vezes chamado DO lógico ou DO simples)

```

[nome:] DO
           bloco
        END DO [nome]

```

A maneira de terminá-lo é com o comando *EXIT*
 EXIT [nome]

Um comando relacionado é o *CYCLE*
 CYCLE [nome]

que transfere o controle para o *END DO* correspondente.

Este tipo de DO é útil quando não sabemos quantas vezes desejamos repetir o laço, mas temos um critério lógico para sua conclusão.

RANDOM_NUMBER (HARVEST)

Descrição: Retorna um número pseudoaleatório ou um array de números pseudoaleatórios com a distribuição uniforme ao longo do intervalo de $0 \leq x < 1$.

Classe: Sub-rotina

Argumento: HARVEST deve ser do tipo REAL. Pode ser array ou escalar. É um argumento INTENT (OUT).

Exemplos

```
REAL X, Y (10, 10)
```

```
! Inicializar X com um numero  
! pseudoaleatorio
```

```
CALL RANDOM_NUMBER (HARVEST = X)
```

```
CALL RANDOM_NUMBER (Y)
```

```
! X e Y contem numeros aleatorios  
! uniformemente distribuidos
```

lista_quadrados_dinamico.f90

Este exemplo introduz a característica da linguagem

- Dimensionamento dinâmico de array

```
PROGRAM lista_quadrados_dinamico  
! Exemplo de dimensionamento  
! dinamico de array  
  
IMPLICIT none  
INTEGER, allocatable :: quadrados( : )  
INTEGER :: numero, quantidade_numeros  
  
! Ache quantos quadrados sao requeridos  
PRINT *, "Quantos quadrados voce quer?"  
READ *, quantidade_numeros  
  
! Aloque espaco suficiente para o  
! numero requerido de quadrados  
ALLOCATE( &  
quadrados( quantidade_numeros ) )  
  
! Armazene os quadrados  
DO numero = 1, quantidade_numeros  
quadrados( numero ) = numero ** 2  
END DO  
  
! Imprima a lista de quadrados  
IF( quantidade_numeros == 1 ) THEN  
PRINT *, "O primeiro quadrado eh: ", &  
quadrados( 1 )  
  
ELSE  
PRINT *, "Os primeiros ", &  
quantidade_numeros, " &
```

```

                                quadrados sao: ", &
                                ( quadrados(numero), numero &
                                = 1, quantidade_numeros )

END IF

! Libere a memoria quando nao for
!   mais necessaria
DEALLOCATE( quadrados )

END PROGRAM lista_quadrados_dinamico

```

```

$ gfortran -o lista_quadrados_dinamico /
> lista_quadrados_dinamico.f90
$ ./lista_quadrados_dinamico
Quantos quadrados voce quer?
10

```

```

Os primeiros      10  quadrados sao:
1                4      9
                16      25      36
49              64      81
100

```

```

$ ./lista_quadrados_dinamico
Quantos quadrados voce quer?
500000000
Fortran runtime error: ALLOCATE: Out of
memory.
$ ./lista_quadrados_dinamico
Quantos quadrados voce quer?
1000000000
Fortran runtime error: Attempt to allocate
negative amount of memory. Possible integer
overflow

```

lista_quadrados_dinamico_verifica.f90

Este exemplo introduz a característica da linguagem

- Resultado do dimensionamento dinâmico de array

```

PROGRAM lista_quadrados_dinamico_verifica
! Exemplo de dimensionamento
!   dinâmico de array com verificacao

IMPLICIT none
INTEGER, allocatable :: quadrados( : )
INTEGER :: numero, quantidade_numeros
INTEGER :: situacao_da_alocacao

! Ache quantos quadrados sao requeridos
PRINT *, "Quantos quadrados voce quer?"
READ *, quantidade_numeros

IF( quantidade_numeros >= 1 ) THEN
! Aloque espaco suficiente para o
!   numero requerido de quadrados
ALLOCATE( &
quadrados( quantidade_numeros ), &
stat= situacao_da_alocacao )

! Verifique se o espaco foi alocado
IF(situacao_da_alocacao == 0 ) THEN

!   Armazene os quadrados
DO numero = 1, quantidade_numeros
quadrados( numero ) = numero ** 2
END DO

!   Imprima a lista de quadrados

```

```

        IF( quantidade_numeros == 1 ) THEN
            PRINT *, "O primeiro &
                &quadrado eh: ", quadrados( 1 )
        ELSE
            PRINT *, "Os primeiros ", &
                quantidade_numeros, &
                " quadrados sao: ", &
                ( quadrados(numero), numero &
                = 1, quantidade_numeros )
        END IF

! Libere a memoria quando nao for
!   mais necessaria
        DEALLOCATE( quadrados )

        ELSE
            PRINT *, "Nao foi possivel alocar &
                &a memoria para os quadrados"

        END IF
    ELSE
        PRINT *, quantidade_numeros, &
            " nao eh um numero valido!"
    END IF

END PROGRAM &
        lista_quadrados_dinamico_verifica

```

```

$ gfortran -o \
> lista_quadrados_dinamico_verifica \
> lista_quadrados_dinamico_verifica.f90
$ ./lista_quadrados_dinamico_verifica
Quantos quadrados voce quer?
10

```

```

Os primeiros           10 quadrados sao:
1                4                9
                16                25                36
49                64                81
100
$ ./lista_quadrados_dinamico_verifica
Quantos quadrados voce quer?
5000000000
Nao foi possivel alocar a memoria para os
quadrados

```

A forma geral do comando **ALLOCATE** é

ALLOCATE (*lista-de-alocação* [, **STAT** = *stat*])

onde *lista-de-alocação* é uma lista de alocações da forma

objeto-alocável [(*lista-de-limites-de-array*)]

cada limite de array tem a forma

[*limite-inferior*.] *limite-superior*

e *stat* é uma variável inteira escalar.

Se o especificador **STAT** = estiver presente, a variável *stat* receberá o valor zero após uma alocação bem sucedida ou um valor positivo no caso da alocação ser mal sucedida.

matriz_produto.f90

Este exemplo introduz as características da linguagem

- Arrays bidimensionais

```
PROGRAM matriz_produto
! Exemplo de um array bidimensional
  IMPLICIT none
  INTEGER, PARAMETER :: tamanho_maximo = 12
! Matriz quadrada
  INTEGER :: matriz( tamanho_maximo, &
                     tamanho_maximo )
  INTEGER :: tamanho_matriz, linha, coluna

! Ache o tamanho requerido para a matriz
  PRINT *, "Qual eh o tamanho da matriz?"
  READ *, tamanho_matriz

  IF( tamanho_matriz <= tamanho_maximo ) &
                                     THEN
!    Preencha a matriz
col: DO coluna = 1, tamanho_matriz
  lin: DO linha = 1, tamanho_matriz
    matriz( linha, coluna ) = &
                                linha * coluna
  END DO lin
END DO col

! Imprima a matriz uma linha de cada vez
DO linha = 1, tamanho_matriz
  PRINT *,( matriz( linha, coluna ), &
            coluna = 1, tamanho_matriz )
END DO
```

```
ELSE
  PRINT *, "Este valor eh muito grande, &
          & o maximo eh ", tamanho_maximo
END IF
```

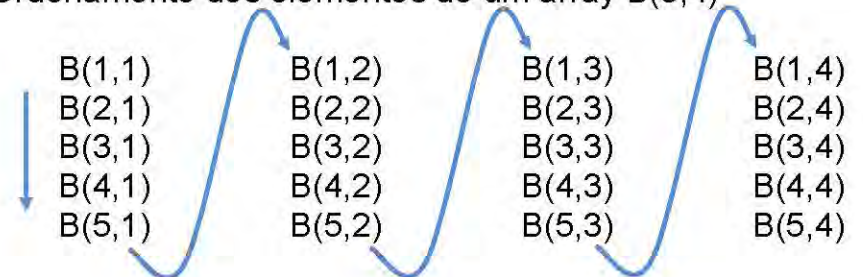
```
END PROGRAM matriz_produto
```

```
$ gfortran -o matriz_produto \
> matriz_produto.f90
$ ./matriz_produto
Qual eh o tamanho da matriz?
3
```

1	2	3
2	4	6
3	6	9

```
$ ./matriz_produto
Qual eh o tamanho da matriz?
13
Este valor eh muito grande, o maximo eh
12
```

Ordenamento dos elementos de um array B(5,4)



```
REAL, DIMENSION(5,4) :: b
```

declara um array de duas dimensões e

```
REAL :: grade(-10:5,-20:-1,0:1,2,2,2)
```

declara sete dimensões. O tamanho do segundo array é

$$16 \times 20 \times 2 \times 2 \times 2 \times 2 \times 2 = 10240$$

O número de dimensões de um array é chamado de *ordem*. Escalares são considerados arrays de ordem zero. O número de elementos em uma dimensão é chamado de *extensão* naquela dimensão. Então, `grade` tem extensões, 16, 20, ...

A sequência de extensões é conhecida como *forma*. Por exemplo, `grade` tem a forma (16, 20, 2, 2, 2, 2).

Também podemos nos referir a uma seção de um array de dimensão maior que um. No caso de `b` acima, por exemplo, `b(2:4,3)`.

matriz_produto_dinamica.f90

Este exemplo introduz as características da linguagem

- **Alocação dinâmica de arrays bidimensionais**

```
PROGRAM matriz_produto_dinamica
! Exemplo de dimensionamento dinamico
! de array bidimensional

IMPLICIT none
INTEGER, ALLOCATABLE :: matriz( :, : )
INTEGER :: tamanho_matriz, linha, coluna
INTEGER :: situacao_da_alocacao

! Ache o tamanho requerido para a matriz
PRINT *, "Qual eh o tamanho da matriz?"
READ *, tamanho_matriz

! Aloque espaco suficiente para a matriz
ALLOCATE( matriz( tamanho_matriz,&
                  tamanho_matriz ), &
          stat=situacao_da_alocacao )

! Verifique se o espaco foi alocado
IF(situacao_da_alocacao == 0 ) THEN

!   Preencha a matriz
col: DO coluna = 1, tamanho_matriz
    lin: DO linha = 1, tamanho_matriz
        matriz( linha, coluna ) = &
            linha * coluna
    END DO lin
END DO col
```

```

!      Imprima a matriz uma linha de cada vez
      DO linha = 1, tamanho_matriz
        PRINT *,( matriz( linha, coluna ),&
                  coluna = 1, tamanho_matriz )
      END DO

! Libere a memoria quando nao for
! mais necessaria
      DEALLOCATE( matriz )

ELSE

      PRINT *, "Nao foi possivel alocar a &
               &memoria para matriz."

END IF

END PROGRAM matriz_produto_dinamica

```

```

$ gfortran -o matriz_produto_dinamica \
> matriz_produto_dinamica.f90
$ ./matriz_produto_dinamica
Qual eh o tamanho da matriz?
3
      1          2          3
      2          4          6
      3          6          9

$ ./matriz_produto_dinamica
Qual eh o tamanho da matriz?
50000000
Nao foi possivel alocar a memoria para matriz

```

Exercícios 3.3

Exercício 3.3.1

Modifique `lista_quadrados4.f90` para pedir também o primeiro número cujo quadrado é requerido (em vez de começar em 1). Acrescente verificações apropriadas nos dados de entrada.

Exercício 3.3.2 (Extra)

Escreva um programa para determinar os números primos até um limite fornecido pelo usuário, usando o crivo de Eratóstenes.

Exercício 3.3.3

Modifique `loteria.f90` para que ele imprima, além dos 6 números sorteados, um 7º número “bônus” (que deve ser diferente do resto).

Exercício 3.3.4 (Extra)

Escreva um programa para aceitar três comprimentos e informar sobre se estes comprimentos poderiam definir um triângulo equilátero, isósceles ou escaleno (3, 2 ou 0 lados iguais) ou se eles não podem formar um triângulo.

Demonstre que o programa funciona, classificando o seguinte:

(1,1,1)	(1,1,0)	(3,2,1)
(2,2,1)	(3,4,5)	(1,2,4)

Dica: Se três comprimentos de formar um triângulo, então duas vezes o lado mais longo deve ser menor do que a soma dos três lados.

3.4 ACESSO A ARQUIVOS

loteria_sorteie_e_lembre.f90

Este exemplo introduz as características da linguagem

- OPEN
- WRITE
- CLOSE

```
PROGRAM loteria_sorteie_e_lembre
! Modifique o programa de sorteio de loteria
!   introduzindo os comandos de acesso
!   a arquivo OPEN, WRITE e CLOSE.

IMPLICIT none
INTEGER, PARAMETER :: total_numeros = 6
INTEGER :: sorteio_numeros(total_numeros)
INTEGER :: numeros_sorteados = 0
INTEGER :: proxima_escolha, isorteio
INTEGER, PARAMETER :: &
    historia_da_loteria = 7
    ! NAO use 5 ou 6

REAL :: xaleatorio
LOGICAL :: diferente
! Para administrar semente do GNA
INTEGER :: i_semente, tamanho_semente, &
    relogio
INTEGER, dimension(:), allocatable :: &
    semente

! Determinar o tamanho da semente
CALL random_seed(size = tamanho_semente)
ALLOCATE(semente(tamanho_semente))
```

```
! Determinar a hora do sistema
CALL system_clock(count=relogio)

! Initialize a sequencia de numeros
!   aleatorios caso contrario teremos
!   os mesmos numeros todas as vezes.
Semente = relogio + &
    37*(/ (i_semente-1, i_semente = 1, &
        tamanho_semente) /)
CALL random_seed(put = semente)
DEALLOCATE(semente)

! Repita ate termos 6 numeros distintos
!   entre 1 e 49
DO
    IF( numeros_sorteados == &
        total_numeros ) EXIT
    CALL random_number( xaleatorio )

    ! xaleatorio eh um numero aleatorio
    !   entre 0.0 e 1.0 entao
    !   multiplique-o por 49 ara obter um
    !   numero aleatorio entre 0 e 49.

    xaleatorio = xaleatorio * 49.0

    ! arredonde para baixo e some 1
    ! para obter um numero inteiro
    ! entre 1 e 49
    proxima_escolha = &
        floor( xaleatorio ) + 1

    ! veja se eh diferente dos numeros
    !   selecionados ate agora e se for
```

```

!   adicione-o a lista
diferente = .true.
DO isorteio = 1, numeros_sorteados
  IF( proxima_escolha == &
      sorteio_numeros&
      ( isorteio ) ) THEN
    diferente = .false.
    EXIT
  END IF
END DO
IF( diferente ) THEN
  numeros_sorteados = &
      numeros_sorteados + 1
  sorteio_numeros&
      ( numeros_sorteados ) =&
      proxima_escolha
END IF

END DO

! Mesmo que PRINT:
WRITE (6,*) "Os numeros da loteria &
           &sao: ",sorteio_numeros

! Registre o sorteio de loteria em um
!   arquivo historico

OPEN( unit= historia_da_loteria,&
      file="historia_da_loteria.txt",&
      status="old", action="write",&
      position="append" )
WRITE( unit= historia_da_loteria, &
      fmt=* ) sorteio_numeros
CLOSE( unit= historia_da_loteria )
END PROGRAM loteria_sorteie_e_lembre

```

```

$ gfortran -o loteria_sorteie_e_lembre.out \
loteria_sorteie_e_lembre.f90
$ ./loteria_sorteie_e_lembre.out
Os numeros da loteria sao:          42
3          36          34
      14          23
$ ./loteria_sorteie_e_lembre.out
Os numeros da loteria sao:          5
6          8          18
      20          31
$ ./loteria_sorteie_e_lembre.out
Os numeros da loteria sao:          33
42          23          16
      7          20
$ cat historia_da_loteria.txt
42          3          36          34          14          23
5           6           8          18          20          31
33          42          23          16           7          20

```

RANDOM_SEED ([SIZE=] [, PUT=] [, GET=])

Descrição: Muda ou pede a semente (ponto de partida) do gerador de números aleatórios usado por RANDOM_NUMBER.

Classe: Sub-rotina

Argumentos: Não mais de um argumento pode ser especificado.

Se nenhum argumento for especificado, será atribuído à semente um valor dependente do processador.

Os três argumentos opcionais seguem:

SIZE Deve ser escalar do tipo INTEGER padrão com INTENT(OUT). Especifica o tamanho mínimo do array usando com os argumentos *PUT* e *GET*.

PUT Deve ser array de uma dimensão do tipo INTEGER padrão. É INTENT(IN) e seu tamanho deve ser maior ou igual ao valor retornado pelo argumento *SIZE*. É usado para reinicializar o valor da semente.

GET Deve ser array de uma dimensão do tipo INTEGER padrão. É INTENT(OUT) e seu tamanho deve ser maior ou igual ao valor retornado pelo argumento *SIZE*. É atribuído o valor atual da semente.

Exemplos

Será atribuído à semente um valor dependente do processador.

CALL RANDOM_SEED

Retorna em **M** o tamanho do array.

CALL RANDOM_SEED (SIZE = M)

Inicializa a semente com o valor contido no array **SEED**.

CALL RANDOM_SEED (PUT = SEED (1 : M))

Lê o valor atual da semente.

CALL RANDOM_SEED (GET = OLD (1 : M))

SYSTEM_CLOCK([COUNT=] [, RATE=] [, MAX=])

Descrição: Retorna o valor atual do relógio do sistema. O resultado exato depende do processador.

Classe: Sub-rotina

Argumentos: Os três argumentos são opcionais:

- | | |
|----------------|--|
| COUNT
(opt) | Deve ser escalar do tipo INTEGER padrão com INTENT(OUT). Contém um valor dependente do processador baseado no relógio do sistema. No gfortran é o número de milisegundos desde Epoch (00:00:00 UTC, January 1, 1970) modulo COUNT_MAX. |
| RATE
(opt) | Deve ser escalar do tipo INTEGER padrão com INTENT(OUT). Contém o número de contagens do relógio por segundo. |
| MAX
(opt) | Deve ser escalar do tipo INTEGER padrão com INTENT(OUT). Contém o maior valor que COUNT pode assumir. |

OPEN([UNIT=] *u* [, *lista_opcional*])

A instrução OPEN estabelece uma conexão entre uma unidade e um arquivo externo e determina as propriedades de conexão. Para realizar a transferência de dados (leitura e escrita), o arquivo deve ser conectado com uma instrução OPEN ou préconectado pelo processador.

u = expressão escalar inteira (não negativo, geralmente de 1-99)

Alguns dos componentes da lista

- | | |
|-----------------------------|---|
| ACTION= <i>act</i> | <i>act</i> é uma expressão tipo caráter que provê o valor READ, WRITE ou RE-ADWRITE (maiúsculas ou minúsculas) |
| FILE= <i>fln</i> | <i>fln</i> é uma expressão tipo caráter |
| Iostat= <i>ios</i> | <i>ios</i> variável inteira escalar, conterà zero se o comando for bem sucedido ou valores positivos caso contrário |
| POSITION= <i>pos</i> | <i>pos</i> é uma expressão tipo caráter que provê o valor ASIS , REWIND ou APPEND |
| STATUS= <i>st</i> | <i>st</i> é uma expressão tipo caráter que provê o valor OLD, NEW, REPLACE, SCRATCH ou UNKNOWN |

CLOSE([UNIT=] *u* [, *lista_opcional*])

u = expressão escalar inteira (não negativo, geralmente de 1-99)

Alguns dos componentes da lista

- | | |
|---------------------------|---|
| Iostat= <i>ios</i> | <i>ios</i> variável inteira escalar, conterà zero se o comando for bem sucedido ou valores positivos caso contrário |
| STATUS= <i>st</i> | <i>st</i> é uma expressão tipo caráter que provê o valor KEEP ou DELETE |

READ ([UNIT=] *u*, [FMT=]*fmt* [,IOSTAT=*ios*]) [*lista*]

WRITE ([UNIT=] *u*, [FMT=]*fmt* [,IOSTAT=*ios*]) [*lista*]

u = expressão escalar inteira

Alguns dos componentes da lista

[FMT=]*fmt* Rótulo de um comando FORMAT; expressão tipo caráter com uma especificação de formato; *

IOSTAT=*ios* Negativo=fim de arquivo, positivo=erro

Forma muito usada:

OPEN(7, FILE="arquivo.dat")

Equivale a

OPEN(7, FILE="arquivo.dat", POSITION="ASIS",&
STATUS="UNKNOWN")

loteria_frequencia_sorteiro.f90

Este exemplo introduz as características da linguagem

- Ler de um arquivo
- END no READ

```
PROGRAM loteria_frequencia_sorteiro
! Mostra o numero de vezes que cada
!     numero foi sorteado

      IMPLICIT none
! Declarar e zerar o array de frequencia
      INTEGER, PARAMETER :: num_maximo = 49
      INTEGER :: frequencia(num_maximo) = 0
      INTEGER, PARAMETER :: total_numeros = 6
      INTEGER :: sorteio_numeros&
                        (total_numeros), inum
      INTEGER :: isorteio, numero
      INTEGER :: situacao
      INTEGER, PARAMETER :: &
                        historia_da_loteria = 8
                        ! NAO use 5 ou 6

! Abrir arquivo da historia da loteria
!     para leitura
      OPEN( unit= historia_da_loteria, &
            file="historia_da_loteria.txt", &
            status="old", action="read" )
```

```

! Ler um sorteio (6 numeros) por linha do
!   arquivo de historia e
!       atualizar a frequencia
DO
    READ( unit= historia_da_loteria, &
          fmt=*, IOSTAT=situacao) &
          sorteio_numeros

    IF (situacao < 0) EXIT
    DO isorteio = 1, total_numeros
        numero = sorteio_numeros(isorteio)
        frequencia( numero ) = &
            frequencia( numero ) + 1
    END DO
END DO

CLOSE( historia_da_loteria )

! Imprimir o array de frequencia
PRINT *, "A frequencia da loteria eh"
DO inum = 1, num_maximo
    PRINT *, "O numero ", inum, &
        " foi sorteado ", &
        frequencia( inum ), " vezes"
END DO

END PROGRAM loteria_frequencia_sorteio

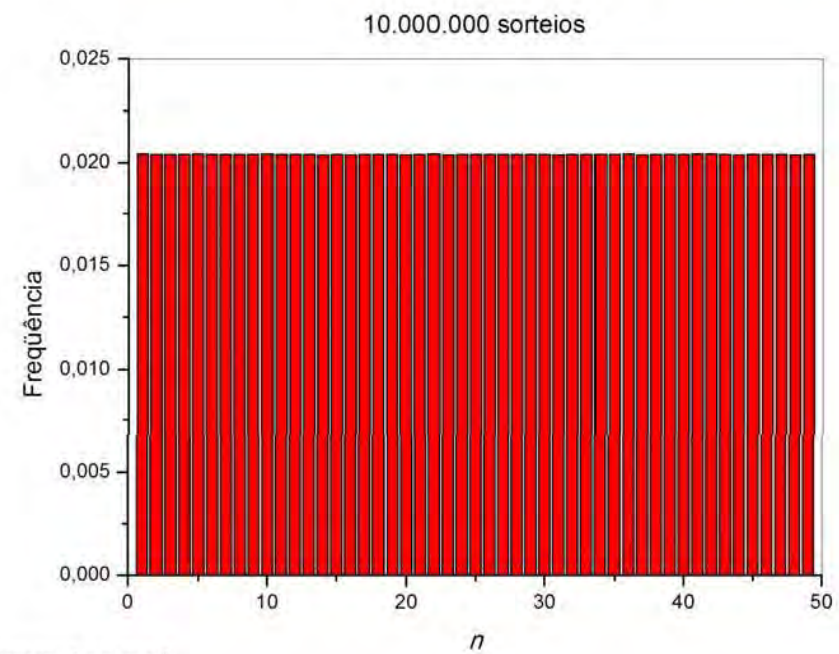
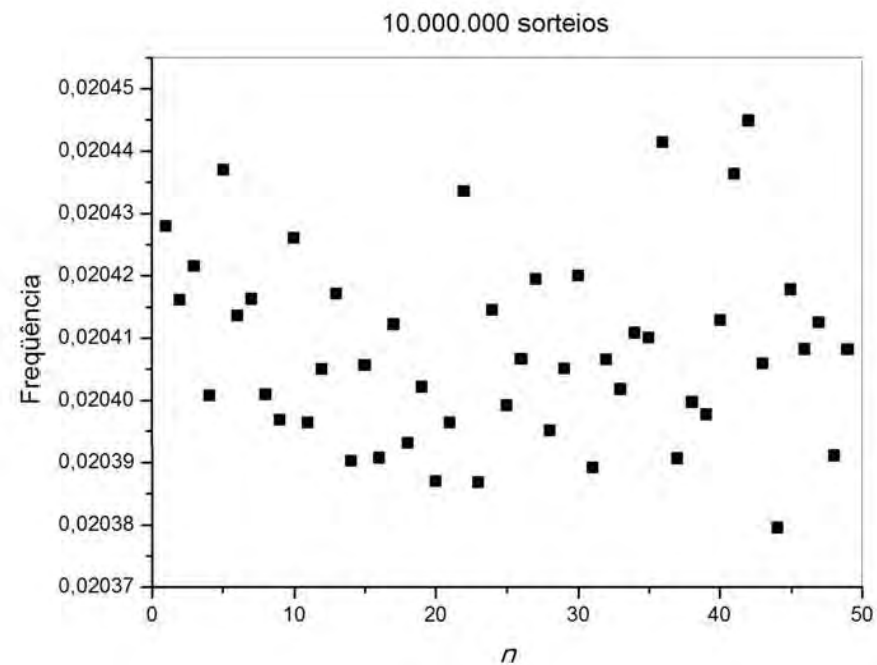
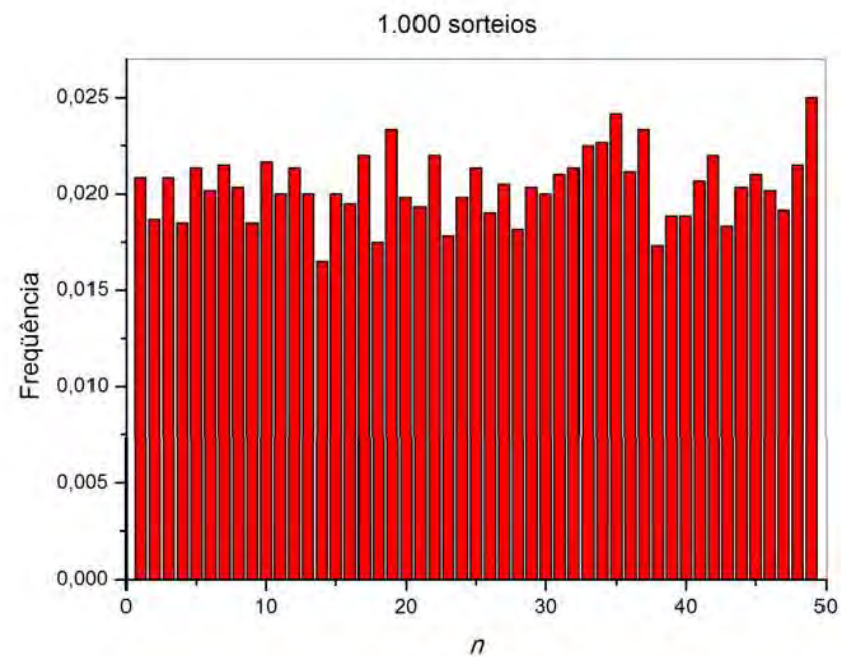
```

```
$ ./loteria_frequencia_sorteio
```

```
A frequencia da Loteria eh
```

0 numero	1	foi sorteado	3	vezes
0 numero	2	foi sorteado	2	vezes
0 numero	3	foi sorteado	3	vezes
0 numero	4	foi sorteado	4	vezes
0 numero	5	foi sorteado	3	vezes
0 numero	6	foi sorteado	3	vezes
0 numero	7	foi sorteado	6	vezes

0 numero	8	foi sorteado	3	vezes
0 numero	9	foi sorteado	1	vezes
0 numero	10	foi sorteado	1	vezes
0 numero	11	foi sorteado	0	vezes
0 numero	12	foi sorteado	1	vezes
0 numero	13	foi sorteado	3	vezes
0 numero	14	foi sorteado	4	vezes
0 numero	15	foi sorteado	4	vezes
0 numero	16	foi sorteado	2	vezes
0 numero	17	foi sorteado	3	vezes
0 numero	18	foi sorteado	2	vezes
0 numero	19	foi sorteado	2	vezes
0 numero	20	foi sorteado	5	vezes
0 numero	21	foi sorteado	4	vezes
0 numero	22	foi sorteado	2	vezes
0 numero	23	foi sorteado	3	vezes
0 numero	24	foi sorteado	2	vezes
0 numero	25	foi sorteado	0	vezes
0 numero	26	foi sorteado	2	vezes
0 numero	27	foi sorteado	1	vezes
0 numero	28	foi sorteado	1	vezes
0 numero	29	foi sorteado	1	vezes
0 numero	30	foi sorteado	1	vezes
0 numero	31	foi sorteado	2	vezes
0 numero	32	foi sorteado	6	vezes
0 numero	33	foi sorteado	3	vezes
0 numero	34	foi sorteado	2	vezes
0 numero	35	foi sorteado	5	vezes
0 numero	36	foi sorteado	0	vezes
0 numero	37	foi sorteado	0	vezes
0 numero	38	foi sorteado	3	vezes
0 numero	39	foi sorteado	3	vezes
0 numero	40	foi sorteado	4	vezes
0 numero	41	foi sorteado	0	vezes
0 numero	42	foi sorteado	2	vezes
0 numero	43	foi sorteado	3	vezes
0 numero	44	foi sorteado	3	vezes
0 numero	45	foi sorteado	3	vezes
0 numero	46	foi sorteado	3	vezes
0 numero	47	foi sorteado	2	vezes
0 numero	48	foi sorteado	2	vezes
0 numero	49	foi sorteado	2	vezes



Exercícios 3.4

Exercício 3.4.1

Usando o programa `loteria_sorteie_e_lembre.f90` várias vezes gere o arquivo `historia_da_loteria.txt`. Modifique o programa `loteria_frequencia_sorteio.f90` para escrever a informação da frequência em um arquivo em vez de imprimi-la na tela.

Exercício 3.4.2

Usando o programa `loteria_sorteie_e_lembre.f90` gere dados de vários sorteios. Modifique o programa `loteria_frequencia_sorteio.f90` para computar a quantidade de números sorteados em cada uma das 5 categorias: 1 a 9, 10 a 19, 20 a 29, 30 a 39 e 40 a 49. Adicione o texto apropriado para explicar a informação impressa. Esta informação é uma estatística enganosa? Se for modifique devidamente o programa. Envie os programas e os arquivos de dados.

Nota 2 até aqui

3.5 FUNÇÕES E SUB-ROTINAS

`area_circulo.f90`

Este exemplo introduz as características da linguagem

- Módulos
- FUNCTION definida pelo usuário
- Atributo INTENT

```
! Modulo contendo dados e functions globais
MODULE funcao_area

CONTAINS

! Sub-programa do modulo
! Definicao da funcao

! Exemplo de uma FUNCTION definida
! pelo usuario
REAL FUNCTION circulo_area( r )
! Calcula a area de um circulo dado seu raio
IMPLICIT none
! Parametro formal
REAL, intent(in) :: r
! Declaracoes locais
REAL, PARAMETER :: pi = 3.14159

    circulo_area = pi * r**2
END FUNCTION circulo_area
END MODULE funcao_area
```

```

PROGRAM area_circulo
! Programa principal para calcular a area de
!   um circulo dado ser raio.

USE funcao_area
IMPLICIT none
REAL :: raio

PRINT *, "Digite o raio do circulo"
READ *, raio
! Verificar a validade
IF( raio >= 0.0 ) then
    PRINT *
    PRINT *, "A area eh ", &
                circulo_area( raio )
ELSE
    PRINT *
    PRINT *, raio, " eh invalido."
END IF
PRINT *
PRINT *, "A area eh ", &
                circulo_area( raio )
! Fim do programa principal
END PROGRAM area_circulo

```

```

$ gfortran -o area_circulo.out \
> area_circulo.f90
$ ./area_circulo.out
  Digite o raio do circulo
-1.25

  A area eh    4.9087343
$ gfortran -o area_circulo3 area_circulo3.f90
$ ./area_circulo3

```

```

  Digite o raio do circulo
-1.25

  -1.25000000    eh invalido.
./area_circulo2
  Digite o raio do circulo
7.58

```

A area eh 180.5045

INTENT (*intent-espec*) *intent-espec* = IN, OUT ou INOUT.

quadrado_e_cubo.f90

Este exemplo introduz a característica da linguagem

- SUBROUTINE

```
! Modulo contendo uma sub-rotina
MODULE subrotina_2_3

CONTAINS

! Subprograma do modulo
! Inicio da definicao da sub-rotina
  SUBROUTINE quadrado_e_cubo( num, &
                             num_quadrado, num_cubo )

    IMPLICIT none
    INTEGER, intent(in) :: num
    INTEGER, intent(out) :: num_quadrado, &
                             num_cubo

    num_quadrado = num*num
    num_cubo = num_quadrado*num

  END SUBROUTINE quadrado_e_cubo
END MODULE subrotina_2_3

PROGRAM exemplo_subrotina
! Programa para demonstrar um exemplo de
! sub-rotina
  USE subrotina_2_3

  IMPLICIT none
  INTEGER :: numero_inteiro, quadrado, cubo
```

```
PRINT *, "Digite um numero inteiro e &
                                     &tecle ENTER"

READ *, numero_inteiro

CALL quadrado_e_cubo( numero_inteiro, &
                     quadrado, cubo )

PRINT *
PRINT *, numero_inteiro, &
        " ao quadrado eh ", quadrado
PRINT *
PRINT *, numero_inteiro, " ao cubo eh ", &
        cubo

! Fim do programa principal
END PROGRAM exemplo_subrotina
```

```
$ gfortran -o exemplo_subrotina \
> exemplo_subrotina.f90
$ ./exemplo_subrotina
Digite um numero inteiro e tecle ENTER
8
```

```
8 ao quadrado eh 64
8 ao cubo eh 512
```

ordenar.f90

Este exemplo introduz as características da linguagem

- Arrays como argumentos de sub-rotinas
- Argumentos que são entrada e saída

```
MODULE sub_ordene

CONTAINS

    SUBROUTINE ordene ( numeros )
        IMPLICIT none
    ! Parametros formais
    !      (array de tamanho presumido)
        INTEGER, intent(inout) :: &
                                numeros ( : )

    ! Variaveis locais
        INTEGER :: tamanho
        INTEGER :: inum, temp
        LOGICAL :: ordenado

    !Obter tamanho do array
        tamanho = size (numeros)
    ! Este algoritmo eh chamado Bubble Sort
        DO
            ordenado = .true.
            DO inum = 2, tamanho
                IF( numeros( inum ) < &
                    numeros( inum-1 ) ) THEN
                    temp = numeros( inum )
                    numeros( inum ) = &
                                numeros( inum-1 )
                    numeros( inum-1 ) = temp
                ordenado = .false.
            END DO
        END DO

END SUBROUTINE ordene

END MODULE sub_ordene
```

```
        END IF
    END DO
    IF( ordenado ) EXIT
END DO

END SUBROUTINE ordene
END MODULE sub_ordene

PROGRAM ordenar
    USE sub_ordene
    IMPLICIT none
    INTEGER :: numeros(5), mais_numeros(10)

    PRINT *, "Digite 5 numeros inteiros e&
              & tecle ENTER"

    READ *, numeros

    CALL ordene( numeros )
    PRINT *, "Os numeros em ordem ascendente&
              & sao ", numeros

    PRINT *
    PRINT *, "Digite outros 10 numeros &
              &inteiros e tecle ENTER"

    READ *, mais_numeros

    CALL ordene ( mais_numeros )
    PRINT *, "Os numeros em ordem ascendente&
              & sao ", mais_numeros

END PROGRAM ordenar
```

```

$ gfortran -o ordenar ordenar.f90
$ ./ordenar
  Digite 5 numeros inteiros e tecle ENTER
5 3 4 6 5
  Os numeros em ordem ascendente sao
3           4           5           5
6

  Digite outros 10 numeros inteiros e tecle
ENTER
12 515 554 354 31 3451 545 3541 5542 21654
  Os numeros em ordem ascendente sao
12           31           354           515
545           554           3451          3541
5542          21654

```

Função Intrínseca SIZE

SIZE (ARRAY[, DIM])

retorna um inteiro escalar padrão que é o tamanho do array **ARRAY**, se **DIM** não for dado, ou a extensão na dimensão **DIM** se o inteiro escalar **DIM** estiver presente.

Exemplo:

```

INTEGER :: numeros(3,4)
PRINT *, SIZE(numeros) ! = 12
PRINT *, SIZE(numeros,1) ! = 3
PRINT *, SIZE(numeros,2) ! = 4

```

operacoes_circulo.f90 usa_operacoes_circulo.f90

Este exemplo introduz a característica da linguagem

- Dados globais
- Compilação em separado

```

! Modulo contendo dados e functions globais
MODULE operacoes_circulo

! Dados globais
REAL, PARAMETER :: pi = 3.14159
INTEGER :: numero_de_chamadas = 0

CONTAINS

! Sub-programas do modulo
REAL FUNCTION circulo_area( r )

    IMPLICIT none
    REAL, intent(in) :: r

    IF( r > 0.0 ) THEN
        circulo_area = pi * r**2
    ELSE
        circulo_area = 0.0
    END IF
    numero_de_chamadas = &
        numero_de_chamadas + 1

END FUNCTION circulo_area

```

```

REAL FUNCTION circunferencia_circulo( r )

    IMPLICIT none
    REAL, intent(in) :: r

    IF( r > 0.0 ) THEN
        circunferencia_circulo = 2.0*pi*r
    ELSE
        circunferencia_circulo = 0.0
    END IF
    numero_de_chamadas = &
        numero_de_chamadas + 1

END FUNCTION circunferencia_circulo

END MODULE operacoes_circulo

```

```

PROGRAM usa_modulo
! Programa para demonstrar o uso de dados
! e functions em modulo

USE operacoes_circulo
IMPLICIT none
REAL :: raio

PRINT *, "Digite o raio do circulo"
READ *, raio

! Use as functions de modulo
!   circulo_area e circunferencia_circulo
PRINT *

```

```

PRINT *, "A area eh ", &
        circulo_area( raio )

PRINT *
PRINT *, "A circunferencia eh ", &
        circunferencia_circulo( raio )

! Temos acesso a dados globais
!   no modulo tambem
PRINT *
PRINT *, "Pi = ", pi
PRINT *
PRINT *, "Functions foram chamadas ", &
        numero_de_chamadas, " vezes"

END PROGRAM usa_modulo

```

```

$ gfortran -c operacoes_circulo.f90
$ gfortran -o usa_operacoes_circulo.out \
>     usa_operacoes_circulo.f90 \
>     operacoes_circulo.o
$ ./usa_operacoes_circulo.out
  Digite o raio do circulo
10

  A area eh      314.1590

  A circunferencia eh      62.83180

  Pi =      3.141590

  Functions foram chamadas      2 vezes

```

imprime_numeros.f90

argumentos_errados_principal.f90

Este exemplo introduz a característica da linguagem

- **Verificação de argumento nas chamadas de funções em modulo**

```
MODULE impressao

CONTAINS

  SUBROUTINE imprime_numeros( r, i )

    IMPLICIT none
    REAL, intent(in) :: r
    INTEGER, intent(in) :: i

    PRINT *, "O numero real eh ", r
    PRINT *, "O numero inteiro eh ", i

  END SUBROUTINE imprime_numeros

END MODULE impressao
```

```
PROGRAM argumentos_errados_principal
! Programa que demonstra a verificacao de
! argumento quando se chama sub-programas
! em um modulo

USE impressao
IMPLICIT none
REAL :: numero_real
INTEGER :: numero_inteiro
```

```
numero_real = 42.0
numero_inteiro = 42

CALL imprime_numeros(numero_inteiro, &
                     numero_real)

END PROGRAM argumentos_errados_principal
```

```
$ gfortran -c imprime_numeros.f90
$ gfortran -o argumentos_errados \
>     argumentos_errados_principal.f90\
>     imprime_numeros.f90.o
In file argumentos_errados_principal.f90:14

    CALL imprime_numeros(numero_inteiro, &
                          1
Error: Type/rank mismatch in argument 'r' at
(1)
```

argumentos_certos_principal.f90

```
PROGRAM argumentos_certos_principal
! Programa que demonstra a verificacao de
! argumento quando se chama sub-programas
! em um modulo

USE impressao
IMPLICIT none
REAL :: numero_real
INTEGER :: numero_inteiro

numero_real = 42.0
numero_inteiro = 42

CALL imprime_numeros(numero_real, &
                     numero_inteiro)

END PROGRAM argumentos_certos_principal
```

```
$ gfortran -c imprime_numeros.f90
$ gfortran -o argumentos_certos \
>     argumentos_certos_principal.f90\
>     imprime_numeros.o
```

Exercícios 3.5

Exercício 3.5.1

Modifique `matriz_produto_dinamica.f90` para preencher a matriz em uma sub-rotina. Faça-o mais geral possível levando em conta a possibilidade que a matriz não tenha o mesmo número de linhas e colunas.

Exercício 3.5.2

Baseado no programa `loteria.f90` escreva uma sub-rotina que seja capaz de produzir uma quantidade qualquer de números aleatórios diferentes (ao invés do valor fixo 6) e com limites arbitrários (ao invés de 1 a 49). O programa principal vai pedir estes dados ao usuário, fazer as verificações necessárias e imprimir os números sorteados.

Redação alternativa para o **mesmo** exercício.

Escreva uma sub-rotina para selecionar N números inteiros aleatórios diferentes dentro de um intervalo que você especifica como argumentos da sub-rotina. O programa principal deve pedir a quantidade de números e o intervalo ao usuário e fazer as verificações apropriadas. Modifique `loteria.f90` para fazer este exercício.

3.6. OUTROS TIPOS DE DADOS

ordenar_nomes.f90

Este exemplo introduz a característica da linguagem

- Tipo de dados *CHARACTER*

```
MODULE ordena

CONTAINS

    SUBROUTINE ordenar_alfabetico ( nomes )
        IMPLICIT none
        ! Argumentos formais
        ! Aqui eh usado array de tamanho presumido
        CHARACTER(len=*), intent(inout) :: &
            nomes( : )

        ! Variaveis Locais
        INTEGER :: tamanho
        INTEGER :: inum
        CHARACTER(len=LEN(nomes)) :: temp
        LOGICAL :: ordenado

        tamanho = size( nomes )
        DO
            ordenado = .true.
            DO inum = 2, tamanho
                IF( nomes( inum ) < &
                    nomes( inum-1 ) ) THEN
                    temp = nomes( inum )
                    nomes( inum ) = nomes(inum-1)
                    nomes( inum-1 ) = temp
                    ordenado = .false.
                END IF
            END DO
        END DO
```

```
        END DO
        IF( ordenado ) EXIT
    END DO

END SUBROUTINE ordenar_alfabetico
END MODULE ordena

PROGRAM ordenar
    ! Programa para demonstrar o uso de
    ! dados tipo CHARACTER
    USE ordena

    IMPLICIT none
    CHARACTER(len=10) :: nomes(5), &
        mais_nomes(10)

    PRINT *, "Digite 5 nomes e tecle ENTER"
    READ *, nomes

    CALL ordenar_alfabetico( nomes )
    PRINT *, "Os nomes em ordem alfabetica&
        & sao ", nomes

    PRINT *
    PRINT *, "Digite outros 10 e tecle ENTER"
    READ *, mais_nomes

    CALL ordenar_alfabetico ( mais_nomes )
    PRINT *, "Os nomes em ordem alfabetica&
        & sao ", mais_nomes

END PROGRAM ordenar
```

```
$ gfortran -o ordenar_nomes \
> ordenar_nomes.f90
$ ./ordenar_nomes
  Digite 5 nomes e tecle ENTER
raquel marcos miguel susana jorge
  Os nomes em ordem alfabetica sao jorge
marcos      miguel      raquel      susana

  Digite outros 10 e tecle ENTER
janete alan lucia egberto roberto manuel
nilse beatriz ivanildo lais
  Os nomes em ordem alfabetica sao alan
beatriz      egberto      ivanildo      janete      lais
lucia        manuel      nilse        roberto
```

Fortran 90 exige que a ordenação do computador satisfaça as seguintes condições:

- $A < B < C \dots < Y < Z$;
 - $0 < 1 < 2 \dots < 8 < 9$;
 - Espaço $< A$ e $Z < 0$ ou espaço < 0 e $9 < A$;
- e, se letras minúsculas estiveram disponíveis,
- $a < b < c \dots < y < z$;
 - Espaço $< a$ e $z < 0$ ou espaço < 0 e $9 < a$.

Portanto, não há regra sobre se os números precedem ou sucedem as letras, nem sobre a posição de caracteres especiais, exceto que o espaço precede todas as outras sequências.

Atribuição

Se atribuírmos uma constante ou variável caráter de tamanho maior a uma variável de tamanho menor haverá truncamento. No caso contrário, a menor será preenchida com espaços em branco à direita.

```
CHARACTER(len=5) :: palavra
palavra = "palavra" ! "palav"
palavra = "giz" ! "giz "
```

Comparação

Se as cadeias de caracteres tiverem tamanhos diferentes a menor será preenchida com espaços em branco à direita.

Operador CHARACTER //

- Fortran 90 usa // para concatenar duas cadeias de caracteres.
- Se as cadeias de caracteres **A** e **B** tiverem comprimentos m e n , a concatenação **A // B** será uma cadeia de caracteres de comprimento $m + n$.

```
CHARACTER(LEN=4) :: John = "John", &
                                     Sam = "Sam"
CHARACTER(LEN=6) :: Lori = "Lori", &
                                     Reagan = "Reagan"
CHARACTER(LEN=10) :: Ans1, Ans2, Ans3, Ans4
```

```
Ans1 = John // Lori ! Ans1 = "JohnLori"
Ans2 = Sam // Reagan ! Ans2 = "SamReagan"
Ans3 = Reagan // Sam ! Ans3 = "ReaganSam"
Ans4 = Lori // Sam ! Ans4 = "LoriSam"
```

testar_hora_e_data.f90

Este exemplo introduz a característica da linguagem

- Sub-rotina **DATE_AND_TIME**

CALL DATE_AND_TIME ([DATE] [, TIME] [, ZONE] [, VALUES])

Descrição: Retorna dados do relógio de tempo real e data

Classe: Sub-rotina

Argumentos: DATE, TIME, ZONE, VALUES todos opcionais:

DATE
(opt) Deve ser escalar e de tipo caráter padrão e deve ter um comprimento de pelo menos oito para poder conter o resultado completo. É um argumento INTENT (OUT). Os oito caracteres mais a esquerda recebem um resultado da forma *CCAAMMDD*, onde *CCAA* é o o ano, *MM* o mês dentro do ano e *DD* o dia dentro do mês.

TIME
(opt) Deve ser escalar e de tipo caráter padrão e deve ter um comprimento de pelo menos 10, a fim de conter o resultado completo. É um argumento INTENT (OUT). Os 10 caracteres mais a esquerda recebem um resultado da forma *hhmmss.sss*, onde *hh* é a hora do dia, *mm* é o minuto da hora e é *ss.sss* segundos e milésimos de segundo.

ZONE
(opt) Deve ser escalar e de tipo caráter padrão e deve ter um comprimento de pelo menos 5 a fim de conter o resultado completo. É um argumento INTENT (OUT). Os cinco caracteres mais a esquerda recebem um resultado da forma $\pm hhmm$ formulário, onde *hh* e *mm* é a diferença horária em relação ao Tempo Universal Coordenado (UTC) em horas e partes de uma hora, expresso em minutos, respectivamente.

VALUES
(opt) Deve ser uma array do tipo inteiro padrão, de uma dimensão. É um argumento INTENT (OUT). Seu tamanho deve ser pelo menos 8. Os valores retornados são os seguintes:

VALUES ano (por exemplo, 1990);

(1)

VALUES mês do ano;

(2)

VALUES dia do mês;

(3)

VALUES diferença horária em relação ao Tempo Universal Coordenado (UTC), em minutos;

VALUES hora do dia, no intervalo de 0-23;

(5)

VALUES minutos da hora, no intervalo de 0-59;

(6)

VALUES segundos do minuto, no intervalo de 0 a 59;

(7)

VALUES milésimos de segundo, na intervalo de 0 a 999;

(8)

```

PROGRAM testar_hora_e_data
  CHARACTER(8)  :: data
  CHARACTER(10) :: hora
  CHARACTER(5)  :: fuso
  INTEGER, dimension(8) :: values
! Usando argumento posicional
  CALL date_and_time(data,hora,fuso, &
                                     values)

  PRINT *, data, " ", hora, " ", fuso
  PRINT *, values

! Usando argumento palavra chave
  CALL date_and_time( DATE=data, ZONE=fuso)
  CALL date_and_time( TIME=hora)
  PRINT *, data, " ", hora, " ", fuso

  CALL date_and_time( VALUES=values)
  PRINT *, values
END PROGRAM testar_hora_e_data

```

```

gfortran -o testar_hora_e_data.out \
> testar_hora_e_data.f90
$ ./testar_hora_e_data.out
20120425 155432.506 -0300
      2012      4      25
-180      15      54      32
506
20120425 155432.506 -0300
      2012      4      25
-180      15      54      32
506

```

abrir_arquivo.f90

Este exemplo introduz a característica da linguagem

- **Variável CHARACTER para especificar nome de arquivo.**

```

PROGRAM abrir_arquivo
! Exemplo de uso de variavel CHARACTER para
! nome de arquivo variavel no OPEN

  IMPLICIT none
  CHARACTER(len=80) :: nome_arquivo
  CHARACTER(len=20) :: primeira_palavra
  INTEGER, PARAMETER :: unidade_entrada = 7

! Solicite ao usuario o nome do arquivo a
! ser aberto
  PRINT *, "Digite o nome do arquivo a&
                                     & ser aberto"

  READ *, nome_arquivo

! Abrir arquivo
  OPEN( unit=unidade_entrada, &
        file=nome_arquivo, status="old" )

! Ver o que estah no arquivo
  READ( unit=unidade_entrada, fmt=* ) &
                                     primeira_palavra

  PRINT *
  PRINT *, "A primeira palavra no arquivo &
                                     &eh:"

  PRINT *, primeira_palavra
END PROGRAM abrir_arquivo

```

```
$ gfortran -o abrir_arquivo \
> abrir_arquivo.f90
$ ./abrir_arquivo
  Digite o nome do arquivo a ser aberto
manual_fortran_90.txt
```

A primeira palavra no arquivo eh:
Fortran

resposta.f90

Este exemplo introduz as características da linguagem

- Subcadeias de CHARACTER
- Operadores lógicos

```
PROGRAM resposta
! Dados CHARACTER em in interacao
!       do programa
! Apresenta sub-cadeias de character e
!   operadores logicos

  IMPLICIT none
  CHARACTER(len=3) :: sim_nao
  CHARACTER(len=1) :: primeira_letra

! Repita ate o usuario querer parar
DO

  PRINT *, "Quer parar?  &
              &(Digite sim ou nao)"
  READ *, sim_nao
  primeira_letra = sim_nao(1:1)
! Vamos verificar apenas a primeira letra
! Note que faz diferenca se eh
!       maiuscula ou minuscula
  IF( primeira_letra .EQ. "s" .or. &
      primeira_letra .EQ. "S" ) EXIT

  END DO

END PROGRAM resposta
```

```

$ gfortran -o resposta resposta.f90
$ ./resposta
Quer parar? (Digite sim ou nao)
asdf
Quer parar? (Digite sim ou nao)
kk
Quer parar? (Digite sim ou nao)
nao
Quer parar? (Digite sim ou nao)
nnn
Quer parar? (Digite sim ou nao)
sss

```

Subcadeias de CHARACTER

Uma subcadeia é uma parte contígua de uma cadeia de caracteres que tem um ponto inicial e um ponto final dentro da cadeia de caracteres. É possível fazer referência a uma subcadeia de uma variável ou constante escalar tipo caráter. A forma de uma subcadeia é:

cadeia-pai (intervalo-da-subcadeia)

onde cadeia pai é um destes

nome de variável escalar

elemento da um array

constante escalar

e um intervalo da subcadeia é

[posição-inicial] : [posição-final]

A posição inicial e final devem ser expressões inteiras escalares.

Exemplo,

```

CHARACTER (15) nome
nome = "Pedro A. Cabral"
nome(1:5) = "Anita"
PRINT *, nome (10:15) != Cabral

```

Operadores lógicos

Opera- dor	Expres- são lógica	Definição
.not.	.not. p	<i>True</i> se p é <i>false</i> ; <i>false</i> se p é <i>true</i>
.and.	p .and. q	Conjunção: <i>true</i> se p e q são ambos <i>true</i> ; <i>false</i> nos outros casos
.or.	p .or. q	Disjunção: <i>true</i> se p ou q ou ambos são <i>true</i> ; <i>false</i> nos outros casos
.eqv.	p .eqv. q	Equivalência: <i>true</i> se p e q são ambos <i>true</i> ou ambos <i>false</i> ; <i>false</i> nos outros casos
.neqv.	p .neqv. q	Não equivalência: <i>true</i> se um dos p ou q é <i>true</i> e o outro <i>false</i> ; <i>false</i> nos outros casos

Tabela verdade

p	q	.NOT.p	p.AND.q	p.OR.q	p.EQV.q	p.NEQV.q
T	T	F	T	T	T	F
T	F	F	F	T	F	T
F	T	T	F	T	F	T
F	F	T	F	F	T	F

Quando uma expressão lógica contém operadores aritméticos, relacionais e lógicos as operações são executadas na seguinte ordem:

1. Operações aritméticas (e funções)
2. Operações relacionais
3. Operações lógicas com a ordem seguinte: **.not.**, **.and.**, **.or.**, **.eqv.** (ou **.neqv.**)

É claro que esta ordem implícita pode ser alterada pelo uso de parênteses.

Podemos atribuir a uma variável lógica o valor de expressão lógica:

```

REAL :: a,b, x, y
LOGICAL :: cond
.
.
cond = a > b .OR. x < 0.0 . AND. y > 1.0

```

resposta_case.f90

Este exemplo introduz as características da linguagem

- Comando SELECT CASE

```

PROGRAM resposta_case
! Dados CHARACTER em interacao
!      do programa
! Apresenta sub-cadeias de character e
!      operadores logicos e o case selector

      IMPLICIT none
      CHARACTER(len=3) :: sim_nao
      CHARACTER(len=1) :: primeira_letra

! Repita ate o usuario querer parar
DO

      PRINT *, "Quer parar?  &
                  &(Digite sim ou nao)"

      READ *, sim_nao
      primeira_letra = sim_nao(1:1)
! Vamos verificar apenas a primeira letra
! Note que faz diferenca se eh
!      maiuscula ou minuscula
      SELECT CASE ( primeira_letra )
      CASE ('s','S')
          EXIT
      CASE ('n','N')
          CYCLE
      CASE default
          PRINT *, "Nao entendi."
      END SELECT

```

```
END DO
```

```
END PROGRAM resposta_case
```

```
$ gfortran -o resposta_case \  
> resposta_case.f90  
$ ./resposta_case  
Quer parar? (Digite sim ou nao)  
xxx  
Nao entendi  
Quer parar? (Digite sim ou nao)  
kkk  
Nao entendi  
Quer parar? (Digite sim ou nao)  
nnn  
Quer parar? (Digite sim ou nao)  
sss
```

```
[nome:] SELECT CASE (expr)  
    [CASE (seletor) [nome]  
        bloco] ...  
    [CASE DEFAULT [nome]  
        bloco]  
END SELECT [nome]
```

expr. escalar CHARACTER, INTEGER ou LOGICAL

seletor:
valor
valor :
: valor
valor1 : valor2

A forma geral do *seletor* é uma lista de sem superposição valores e intervalos do mesmo tipo que *expr*, por exemplo, **CASE** (1,2,7,10:17,23).

aritmetica_dupla.f90

Este exemplo introduz a característica da linguagem

- **DOUBLE PRECISION**

```
PROGRAM aritmetica_dupla  
! Exemplos de aritmetica precisao dupla  
  
    IMPLICIT none  
    DOUBLE PRECISION :: largura, comprimento, &  
        area, numerador, denominador, quociente  
  
! Multiplicacao  
    largura = 2.3d0  
    comprimento = 5.47d0  
    area = largura * comprimento  
    PRINT *  
    PRINT *, "A area do retangulo com &  
        largura ", largura, " e comprimento ", &  
        comprimento, " eh ", area  
  
! Divisao  
    numerador = 2.20d1  
    denominador = 7.0d-1  
    quociente = numerador / denominador  
    PRINT *  
    PRINT *, "A divisao de ", numerador, &  
        " por ", denominador, " dah ", quociente  
  
END PROGRAM aritmetica_dupla
```

```
$ gfortran -o aritmetica_dupla \
> aritmetica_dupla.f90
$ ./aritmetica_dupla
```

```
A area do retangulo com largura
2.2999999999999998      e comprimento
5.4699999999999998      eh
12.5809999999999998
```

```
A divisao de      220.00000000000000
por      0.6999999999999996      dah
314.28571428571428
```

DOUBLE PRECISION é sinônimo de REAL (KIND = KIND(1.0D0)).

y = DBLE(x) converte real ou inteiro em precisão dupla.

Tipicamente os dados do tipo DOUBLE PRECISION ocupam 64 bits (8 bytes). Eles têm 15 algarismos significativos decimais aproximadamente e o intervalo deles é de

$\pm 2.22507385850720138D-308$ a

$\pm 1.79769313486231571D+308$.

Mecanismo portátil de seleção da precisão

Um processador pode suportar diferentes precisões (ou representações) de INTEGER, REAL e COMPLEX, diferentes conjuntos de caracteres (por exemplo, notação musical, árabe, escrita cirílica) e diferentes maneiras de representar objetos LOGICAL.

Um exemplo da parametrização de um tipo intrínseco é,

```
INTEGER(KIND=1) :: ik
```

```
REAL(4) :: rk
```

Tipo de dados inteiros por KIND

No caso de SELECTED_INT_KIND o argumento especifica o intervalo mínimo de expoente decimal para o modelo desejado. Por exemplo,

```
INTEGER :: curto, medio, longo, mlongo
PARAMETER (curto = SELECTED_INT_KIND(2), &
           medio = SELECTED_INT_KIND(4), &
           longo = SELECTED_INT_KIND(10), &
           mlongo = SELECTED_INT_KIND(100))
```

```
INTEGER(curto) :: a,b,c
```

```
INTEGER(medio) :: d,e,f
```

```
INTEGER(longo) :: g,h,i
```

As declarações acima especificam que a precisão deve ser, no mínimo:

- $(-10^2, 10^2)$. — **curto**
- $(-10^4, 10^4)$. — **medio**
- $(-10^{10}, 10^{10})$. — **longo**
- $(-10^{100}, 10^{100})$. — **mlongo**

Se um modelo com pelo menos a precisão indicada não estiver disponível, a função irá retornar -1 e qualquer declaração com esse tipo de valor dará um erro (em tempo de compilação).

Constantes de Selected Integer Kind

Constantes de um tipo (kind) selecionados são indicados, anexando underscore (traço inferior) seguido do número ou um nome de espécie constante inteira:

```
100_2, 1238_4, 54321_longo
```

A constante

```
1000_curto
```

não é válida.

Seleção de tipo (KIND) Real

O intrínseco `SELECTED_REAL_KIND` pode ser parametrizado com dois valores, a precisão mínima e o intervalo mínimo de expoente decimal, o tipo de valor correspondente será retornado ou, se o intervalo desejado não pode ser suportado, o resultado será -1.

`SELECTED_REAL_KIND (8,9)` retornará um valor `KIND` que suporta números com uma precisão de 8 dígitos e expoente de escala decimal entre -9 e 9.

As declarações são feitas na mesma forma que para números inteiros, por exemplo,

```
INTEGER, PARAMETER :: &  
    r1 = SELECTED_REAL_KIND(5,20), &  
    r2 = SELECTED_REAL_KIND(10,40)  
REAL(KIND=r1) :: x, y, z  
REAL(r2), PARAMETER :: diff = 100.0_r2
```

Variáveis `COMPLEX` são especificadas da mesma forma.

```
COMPLEX(KIND=r1) :: cinema  
COMPLEX(r2) :: &  
    inferioridade = (100.0_r2,99.0_r2)
```

Kind Functions

`KIND` é uma função intrínseca que retorna o inteiro que corresponde ao tipo (`KIND`) do argumento. Por exemplo, `KIND (A)` retornará o número inteiro que corresponde ao tipo (`KIND`) de `A` e `KIND (20)` retorna o valor escolhido para a representação usada para o tipo inteiro padrão.

As funções de conversão de tipo, `INT`, `NINT`, `REAL`, `DBLE`, etc, têm um argumento `KIND=` opcional para indicar a representação do resultado da função, por exemplo,

```
INTEGER(KIND=6) :: ik6  
REAL x  
INTEGER i  
  
...  
ik6 = REAL(x,KIND=6)  
! ou melhor  
ik6 = REAL(x,KIND(ik6))
```

aritmetica_complexa.f90

Este exemplo introduz a característica da linguagem

- Tipo de dado **COMPLEX**

```
PROGRAM aritmetica_complexa
! Exemplo de aritmetica complexa

IMPLICIT none
COMPLEX, PARAMETER :: complexo_menos1 = &
                                (-1.0, 0.0)

COMPLEX :: z

! raiz quadrada complexa
z = SQRT( complexo_menos1 )
PRINT *, "A raiz quadrada de -1 eh ", z

END PROGRAM aritmetica_complexa

$ gfortran -o aritmetica_complexa \
> aritmetica_complexa.f90
$ ./aritmetica_complexa
A raiz quadrada de -1 eh ( 0.000000 ,
1.000000 )
```

CMPLX(x[,y]) converte x ou (x,y) para o tipo COMPLEX.

Exercícios 3.6

Exercício 3.6.1

Usando os conhecimentos que você tem agora, aprimore (torne-o funcionalmente perfeito) o programa do exercício 3.2.2.

Exercício 3.6.2 (Extra)

Execute o programa `ordenar_nomes.f90` e desta vez digite nomes próprios com algumas letras maiúsculas e outras minúsculas. O programa ainda funciona? Se não, modifique-o adequadamente. Os nomes devem ser impressos na ordem alfabética e na sua forma original. **Não** use informações obtidas de outras fontes, por exemplo, internet. *Sugestão:* a função intrínseca **ichar** dá a posição de um caráter na sequência do seu argumento e a de função intrínseca **char** dá o caráter que está na posição **n** onde **n** é o seu argumento. Use também as informações da página 93.

Exercício 3.6.4 (Extra)

Escreva um programa que solicita ao usuário um número entre 1 e 3999 e o converte para números romanos. Este programa só usa um IF (para verificar se o dado de entrada é válido). *Dica:* como faria este programa se fosse para converter algarismos arábicos na forma de caracteres?

3.7. MAIS SOBRE ENTRADA E SAÍDA

abrir_arquivo_antigo_verificar.f90

abrir_arquivo_novo_verificar.f90

Este exemplo introduz a característica da linguagem

- Especificador IOSTAT no comando OPEN

```
PROGRAM abrir_arquivo_antigo_verificar
! Exemplo do uso do IOSTAT para verificar
! o sucesso do comando OPEN

IMPLICIT none
CHARACTER(len=80) :: nome_arquivo, &
                        primeira_palavra
INTEGER, PARAMETER :: &
                        unidade_entrada = 7, ok = 0
INTEGER :: situacao_open

! Solicite ao usuario que digite o nome de
! um arquivo a ser aberto
PRINT *, "Digite o nome do arquivo a ser &
        & lido"
READ *, nome_arquivo

! Use o especificador IOSTAT para ver se
! o OPEN funcionou
OPEN( unit=unidade_entrada, &
      file=nome_arquivo, status="old", &
      iostat=situacao_open )

IF( situacao_open == ok ) THEN
```

```
!   Ver o que estah no arquivo
      READ( unit=unidade_entrada, fmt=* ) &
                        primeira_palavra

      PRINT *
      PRINT *, "A primeira palavra no &
                        & arquivo eh:"
      PRINT *, primeira_palavra

ELSE

      PRINT *
      PRINT *, "Incapaz de abrir arquivo"
END IF

END PROGRAM abrir_arquivo_antigo_verificar
```

```
$ gfortran -o abrir_arquivo_antigo_verificar\
> abrir_arquivo_antigo_verificar.f90
$ ./abrir_arquivo_antigo_verificar
  Digite o nome do arquivo a ser aberto
jsim
```

```
  Incapaz de abrir arquivo
$ ./abrir_arquivo_antigo_verificar
  Digite o nome do arquivo a ser aberto
antigo.dat
```

```
  A primeira palavra no arquivo eh:
  Letra
```

```

PROGRAM abrir_arquivo_novo_verificar
! Exemplo do uso do IOSTAT para verificar
! o sucesso do comando OPEN

IMPLICIT none
CHARACTER(LEN=80) :: nome_arquivo, &
                                primeira_palavra
INTEGER, PARAMETER :: &
                                unidade_saida = 8, ok = 0
INTEGER :: situacao_open

! Solicite ao usuario que digite o nome de
! um arquivo a ser aberto
PRINT *, "Digite o nome do arquivo a ser&
        & criado"
READ *, nome_arquivo

! Use o especificador IOSTAT para ver se
! o OPEN funcionou
OPEN( unit=unidade_saida, &
      file=nome_arquivo, status="new", &
      iostat=situacao_open )

IF( situacao_open == ok ) THEN

!   Escreva algo no arquivo
    WRITE (unit=unidade_saida, fmt=* ) &
        "Escrever no arquivo novo"

ELSE

    PRINT *
    PRINT *, "Incapaz de criar arquivo"

```

```

END IF

END PROGRAM abrir_arquivo_novo_verificar

$ gfortran -o abrir_arquivo_novo_verificar \
> abrir_arquivo_novo_verificar.f90
$ ./abrir_arquivo_novo_verificar
  Digite o nome do arquivo a ser criado
antigo.dat

  Incapaz de criar arquivo
$ ./abrir_arquivo_novo_verificar
  Digite o nome do arquivo a ser criado
novo.livro
$ cat novo.livro
  Escrever no arquivo novo

```

program loteria_sorteie_e_lembre2

Este exemplo introduz

- A maneira correta de guardar a semente

```
PROGRAM loteria_sorteie_e_lembre2
! Modifique o programa de sorteio de loteria
!   introduzindo os comandos de acesso
!   a arquivo OPEN, WRITE e CLOSE.

IMPLICIT none
INTEGER, PARAMETER :: total_numeros = 6
INTEGER :: sorteio_numeros(total_numeros)
INTEGER :: numeros_sorteados
INTEGER :: proxima_escolha, isorteio
INTEGER, PARAMETER :: &
    historia_da_loteria = 7
    ! NAO use 5 ou 6

REAL :: xaleatorio
LOGICAL :: diferente

! Para guardar a semente do
!   gerador de numeros aleatorios
    INTEGER, ALLOCATABLE :: semente ( : )
! numero de elementos do vetor semente
    INTEGER :: k
! Para ver a situação do arquivo de semente
    INTEGER :: situacao

    INTEGER, PARAMETER :: ok = 0, &
        guardar_semente = 8
```

```
! Initialize a sequencia de numeros
!   aleatorios caso contrario teremos
!   os mesmos numeros todas as vezes.

!Obter o tamanho da semente
    CALL random_seed(size = k)
    ALLOCATE (semente(k))

! Ler a semente que foi guardada
    OPEN(guardar_semente,FILE="semente.dat",&
        STATUS="old",ACTION="READ",&
        IOSTAT=situacao)

    IF (situacao == ok) THEN
! Ler a semente anterior e usa-la
        READ (guardar_semente,*) semente
        CLOSE (guardar_semente)
        CALL random_seed (PUT=semente)
    ELSE
! Presumir que o arquivo nao existe
!   e cria-lo
        OPEN (guardar_semente,&
            FILE="semente.dat", STATUS="new")
!   e inicializar o gerador da forma padrao
        CLOSE (guardar_semente)
        CALL random_seed ()
    ENDIF

! Repita ate termos 6 numeros distintos
!   entre 1 e 49
    numeros_sorteados = 0
    DO
        IF( numeros_sorteados == &
            total_numeros ) EXIT
```

```

        CALL random_number( xaleatorio )

! xaleatorio eh um numero aleatorio
!   entre 0.0 e 1.0 entao
!   multiplique-o por 49 ara obter um
!   numero aleatorio entre 0 e 49.

        xaleatorio = xaleatorio * 49.0

! arredonde para baixo e some 1
! para obter um numero inteiro
! entre 1 e 49
        proxima_escolha = &
                        floor( xaleatorio ) + 1

! veja se eh diferente dos numeros
! selecionados ate agora e se for
! adicione-o a lista
        diferente = .true.
        DO isorteio = 1, numeros_sorteados
            IF( proxima_escolha == &
                sorteio_numeros&
                ( isorteio ) ) THEN
                diferente = .false.
                EXIT
            END IF
        END DO
        IF( diferente ) THEN
            numeros_sorteados = &
                                numeros_sorteados + 1
            sorteio_numeros&
                ( numeros_sorteados ) =&
                    proxima_escolha

        END IF

```

```

        END DO
! Mesmo que print:
        WRITE(6,*) "Os numeros da loteria sao: ", &
                    sorteio_numeros

! Registre o sorteio de loteria em um
!   arquivo historico

        OPEN ( unit= historia_da_loteria,&
                file="historia_da_loteria.txt",&
                action="write",position="append" )
        WRITE ( unit= historia_da_loteria, &
                fmt=* ) sorteio_numeros
        CLOSE ( unit= historia_da_loteria )

! Guarde a semente
!   para usa-la da proxima vez
        OPEN (guardar_semente,&
                file="semente.dat",status="old",&
                action="write", position="rewind")
        CALL random_seed (get=semente)
        WRITE (guardar_semente,*) semente
        CLOSE (guardar_semente)

END PROGRAM loteria_sorteie_e_lembre2

```

```

$ ls
loteria_sorteie_e_lembre2.f90
$ gfortran -o loteria_sorteie_e_lembre2.out\
> loteria_sorteie_e_lembre2.f90
$ touch historia_da_loteria.txt
$ ls
historia_da_loteria.txt

```

```

loteria_sorteia_e_lembre2.out
loteria_sorteia_e_lembre2.f90
$ ./loteria_sorteia_e_lembre2.out
  Os numeros da loteria sao:
28          48          37
          19          24
$ ls
historia_da_loteria.txt
loteria_sorteia_e_lembre2.out
loteria_sorteia_e_lembre2.f90
semente.dat
$ cat historia_da_loteria.txt
          49          28          48
37          19          24
$ cat semente.dat
1962653439 1127294890 167739224
1132865128 987654321 458629013 58285
9209 438195021
$ ./loteria_sorteia_e_lembre2.out
  Os numeros da loteria sao:
1          18          17
          11          7
$ cat historia_da_loteria.txt
          49          28          48
37          19          24
          4          1          18
17          11          7
$ cat semente.dat
-189103591 -1480004845 85145367
1316773834 987654321 458629013 58285
9209 438195021

```

lista_quadrados_formatado.f90

Este exemplo introduz a característica da linguagem

- **Descritores de formatação para CHARACTER, INTEGER e nova linha**

```

PROGRAM lista_quadrados_formatado
! Exemplo de saída formatada com descritores
! character, integer e nova linha

      IMPLICIT none
      INTEGER :: numero
      INTEGER :: quadrados(20)

! Armazene os primeiros 20 quadrados
      DO numero = 1, 20
         quadrados( numero ) = numero ** 2
      END DO

! Imprima a lista de quadrados
      PRINT "( a / ( 5i8 ) )", "Os primeiros &
                                &20 quadrados sao: ", &
                                quadrados

END PROGRAM lista_quadrados_formatado

```

```

$ gfortran -o lista_quadrados_formatado \
> lista_quadrados_formatado.f90
$ ./lista_quadrados_formatado
Os primeiros 20 quadrados sao:

```

1	4	9	16	25
36	49	64	81	100
121	144	169	196	225
256	289	324	361	400

```
PRINT "( a40 / ( 5i8 ) )", "Os primeiros&
      &20 quadrados sao: ", &
      quadrados
```

Os primeiros 20 quadrados sao:

1	4	9	16	25
36	49	64	81	100
121	144	169	196	225
256	289	324	361	400

```
PRINT "( a10 / ( 5i8 ) )", "Os primeiros&
      &20 quadrados sao: ", &
      quadrados
```

Os primeir

1	4	9	16	25
36	49	64	81	100
121	144	169	196	225
256	289	324	361	400

```
PRINT "( a / ( 5i3 ) )", "Os primeiros &
      &20 quadrados sao: ", &
      quadrados
```

Os primeiros 20 quadrados sao:

```
1 4 9 16 25
36 49 64 81 100
121 144 169 196 225
256 289 324 361 400
```

```
PRINT "( a / ( 5i2 ) )", "Os primeiros &
      &20 quadrados sao: ", &
      quadrados
```

Os primeiros 20 quadrados sao:

```
1 4 9 16 25
36 49 64 81 100
*****
*****
```

Há três maneiras de se especificar o formato:

1.

```
PRINT '(i10,f10.3,a10)',tipo_int, &
      tipo_real, tipo_char
```

2.

```
CHARACTER(LEN=*) , PARAMETER :: f1 = &
' (i10,f10.3,a10) '
PRINT f1,tipo_int, tipo_real, tipo_char
```

3.

```
PRINT 10,tipo_int, tipo_real, tipo_char
10 FORMAT(i10,f10.3,a10)
```

```
WRITE (6, ' ("Os primeiros 20 quadrados &
      &sao:"/ ( 5i8 ))') quadrados
```

aritmetica_dupla_formatado.f90

Este exemplo introduz a característica da linguagem

- **Descritores de formato REAL (ponto fixo)**

```
PROGRAM aritmetica_dupla_formatado
! Exemplos de especificador do formato
! real ponto-fixo
  IMPLICIT none
  DOUBLE PRECISION :: largura, comprimento, &
    area, numerador, denominador, quociente

! Multiplicacao
  largura = 2.3d0
  comprimento = 5.47d0
  area = largura * comprimento
! Imprimir primeiro usando saida
! com formato livre (list-directed)
  PRINT *, "List-directed:"
  PRINT *, &
    "A area do retangulo com largura " &
    largura, &
    " e comprimento ", comprimento, &
    " eh ", area
! Imprima esta linha com especificador
! de formato
  PRINT *, "Formatado:"
! Esta eh a forma correta de escrever texto
! constante (sem usar o especificador a)
  PRINT '( "A area do retangulo com &
    &largura ", f3.1, " e comprimento ", &
    f4.2, " eh ", f6.3 )', &
    largura, comprimento, area
```

```
! Divisao
  numerador = 22.0d0
  denominador = 7.0d0
  quociente = numerador / denominador
! Imprimir primeiro usando saida
! com formato livre (list-directed)
  PRINT *, "List-directed:"
  PRINT *, &
    "A divisao de ", numerador, &
    " por ", denominador, " dah ", quociente
! Imprima esta linha com especificador
! de formato
  PRINT *, "Formatado:"
! Esta eh a forma correta de escrever texto
! constante (sem usar o especificador a)
  PRINT '( "A divisao de ", f4.1, " por ", &
    f3.1, " dah ", f6.4 )', &
    numerador, denominador, quociente
END PROGRAM aritmetica_dupla_formatado
```

```
$ gfortran -o aritmetica_dupla_formatado \
> aritmetica_dupla_formatado.f90
$ ./aritmetica_dupla_formatado
List-directed
  A area do retangulo com largura
2.3000000000000000          e comprimento
5.4700000000000000          eh
12.581000000000000
Formatado
A area do retangulo com largura 2.3 e
comprimento 5.47 eh 12.581
```

```

List-directed
A divisao de      22.00000000000000      por
7.000000000000000      dah
3.14285714285714
Formatado
A divisao de 22.0 por 7.0 dah 3.1429

```

imprimir_numeros_grandes.f90

Este exemplo introduz a característica da linguagem

- **Descritores de formato REAL (notação científica)**

```

PROGRAM imprimir_numeros_grandes
! Exemplo de especificador de formato
! científico para imprimir numeros que
! tem um grande intervalo de valores.

IMPLICIT none
REAL :: numero_muito_grande = 1.234e10, &
      numero_muito_pequeno = 5.678e-10, &
      numero_moderado = 42.0
REAL :: coluna(3)
INTEGER :: i

coluna(1) = numero_muito_grande
coluna(2) = numero_moderado
coluna(3) = numero_muito_pequeno

! Imprimir primeiro usando saida
! com formato livre (list-directed)
PRINT *, "Coluna impressa com formato&
      & list-directed:"

DO i = 1, 3
  PRINT *, coluna(i)
END DO

! Imprimir usando especificador de
! formato científico
PRINT *
PRINT *, "Coluna impressa com formato&
      & científico:"

```

```
PRINT "( es10,3 )", coluna
```

```
END PROGRAM imprimir_numeros_grandes
```

```
$ gfortran -o imprimir_numeros_grandes \
> imprimir_numeros_grandes.f90
$ ./imprimir_numeros_grandes
Coluna impressa com formato list-directed:
  1.2340000E+10
    42.00000
  5.6779997E-10
```

```
Coluna impressa com formato científico:
1.234E+10
4.200E+01
5.678E-10
```

Descritores de edição

1. Repetição

```
10F12.3
```

```
PRINT '(4(I5,F8.2))', (i(j), a(j), j=1,4)
```

```
PRINT '(I5,F8.2,I5,F8.2,I5,F8.2,I5,F8.2)', &
(i(j), a(j)), j=1,4)
```

```
PRINT '(2(2I5,2F8.2))', &
i(1), i(2), a(1), a(2), i(3), i(4), a(3), a(4)
```

```
PRINT '(10I8)', (i(j), j=1,180)
```

2. Descritores de edição de controle

[n]X	espaço
[n]/	linha

3. Descritores de edição de dados

INTEGER	Ic[.m]			
REAL	Fc.d	F11.4		13.4380
			0.0217	
	Ec.d	E11.4	0.2170E-	0.1344E+02
			01	
	ESc.d	ES11.4	2.1700E-	1.3438E+01
			02	
	ENc.d	EN11.4	21.7000E-	13.4380E+00
			03	
	Gc.d	G11.4	0.2170E-	13.44
			01	
CHARAC- TER	A[c]			
COMPLEX	Pares de F ou E			

E11.4	F11.4	G11.4
-0.1235E-05	-0.0000	-0.1235E-05
0.1235E-05	0.0000	0.1235E-05
-0.1235E-04	-0.0000	-0.1235E-04
0.1235E-04	0.0000	0.1235E-04
-0.1235E-03	-0.0001	-0.1235E-03
0.1235E-03	0.0001	0.1235E-03
-0.1235E-02	-0.0012	-0.1235E-02
0.1235E-02	0.0012	0.1235E-02
-0.1235E-01	-0.0123	-0.1235E-01
0.1235E-01	0.0123	0.1235E-01
-0.1235E+00	-0.1235	-0.1235
0.1235E+00	0.1235	0.1235
-0.1235E+01	-1.2346	-1.235
0.1235E+01	1.2346	1.235
-0.1235E+02	-12.3457	-12.35
0.1235E+02	12.3457	12.35
-0.1235E+03	-123.4568	-123.5
0.1235E+03	123.4568	123.5
-0.1235E+04	-1234.5679	-1235.
0.1235E+04	1234.5679	1235.
-0.1235E+05	-12345.6787	-0.1235E+05
0.1235E+05	12345.6787	0.1235E+05

4. Edição de largura de campo mínima (Fortran 95 apenas)

i0

f0.3

```
WRITE (6, ' ("Os primeiros 20 quadrados &
&sao:"/ (5(i0,1x) ))') quadrados
```

Os primeiros 20 quadrados sao:

```
1 4 9 16 25
36 49 64 81 100
121 144 169 196 225
256 289 324 361 400
```

Arquivos internos

Este é um mecanismo semelhante aos comandos de entrada e saída formatado para arquivos, que permite a conversão de dados numéricos da representação binária interna para a representação "formatada" e vice-versa. Este mecanismo de conversão trata cadeias de caracteres ou arrays destes e os trata como se fossem arquivos. Substituindo o número da unidade pelo nome da variável tipo CHARACTER (escalar ou array) no READ pode-se converter texto em número e vice-versa no WRITE.

```
PROGRAM read_interno
  IMPLICIT none
  INTEGER :: ival(30), chave, i
  CHARACTER(30):: buffer
  CHARACTER(6) :: forma(3) = (/ '(30i1)', &
                                '(15i2)', '(10i3)' /)

  READ (*, '(i1,a30)') chave, buffer
  READ (buffer, forma(chave)) (ival(i), &
                                i=1,30/ chave)

  WRITE (*,*) (ival(i), i=1,30/ chave)
END PROGRAM read_interno
```

```

PROGRAM write_interno
  IMPLICIT none
  INTEGER :: n, m
  CHARACTER(len=15) :: ifmt
  INTEGER :: iarray(5) = &
                          (/ 1, 2, 3, 4, 5 /)

! Pedir as especificacoes do formato
WRITE (6,*) 'Digite o numero de espacos &
           &antes do conteudo do array:'
READ (5,*) n
WRITE (6,*) 'Digite o fator de &
           &repeticao: '
READ (5,*) m

! Criar o arquivo interno
!   contendo o format:
WRITE (ifmt," (1X, '(', I2, 'X,', I2, &
           ' (I2,X)) ')" n, m

! Imprimir o array usando o format no
!   arquivo interno
WRITE (6,*) 'O conteudo do array &
           &"iarray" eh: '
WRITE (6,ifmt) iarray

! Imprimir o formato usado:
WRITE (6,*) 'O format usado para &
           &imprimir o conteudo eh: '
WRITE (6,*) 'FORMAT ', ifmt

END PROGRAM write_interno

```

```

$ gfortran -o write_interno.out \
> write_interno.f90
$ ./write_interno.out
  Digite o numero de espacos antes do con-
teudo do array:
2
  Digite o fator de repeticao:
2
  O conteudo do array "iarray" eh:
    1  2
    3  4
    5
  O format usado para imprimir o conteudo eh:
  FORMAT ( 2X, 2(I2,X))

```

```

CHARACTER (LEN=13) :: arquivo
PRINT *, "Por favor, digite o valor do &
           &raio"
READ *,raio
WRITE (arquivo,' ("RAIO=",F4.2, ".dat") ')&
           raio
OPEN (UNIT=7, FILE= arquivo, &
           STATUS="new")

```

erro_read.f90

Este exemplo introduz a característica da linguagem

- **Especificador ADVANCE no comando WRITE**
- **Especificador IOSTAT no comando READ**

```
PROGRAM erro_read
! Exemplo verificacao de erro no
! comando READ
IMPLICIT none
INTEGER :: quadrados ( 20 )
INTEGER :: numero, quantidade_numeros
INTEGER :: situacao
! Descobrir quantos quadrados sao requeridos
DO
    WRITE (*, ' ("Quantos quadrados voce &
                &quer? " ) ', ADVANCE="NO")

    READ ( unit=*, fmt=*, IOSTAT=situacao ) &
        quantidade_numeros
    IF (situacao == 0) EXIT
    WRITE (*, ' ("Numero invalido. &
                &Tente novamente." ) ')
END DO

! Armazene os quadrados
DO numero = 1, quantidade_numeros
    quadrados ( numero ) = numero ** 2
END DO

! Imprima a lista dos quadrados
PRINT *
PRINT *, "Os primeiros ", &
        quantidade_numeros, &
        " quadrados sao: ", &
```

```
( quadrados(numero), numero = 1, &
    quantidade_numeros )
END PROGRAM erro_read
```

```
$ gfortran -o erro_read2 erro_read2.f90
$ ./erro_read2
Quantos quadrados voce quer? 12.5
Numero invalido. Tente novamente
Quantos quadrados voce quer? usn
Numero invalido. Tente novamente
Quantos quadrados voce quer? 3

Os primeiros          3 quadrados sao:
1                   4          9
```

Exercícios 3.7

Exercício 3.7.1

Modifique `abrir_arquivo_antigo_verificar.f90` para permitir ao usuário um máximo de 3 tentativas de especificar um arquivo a ser aberto. Se houver falha em uma das duas primeiras tentativas o programa deve escrever a mensagem de erro “Arquivo inexistente. Tente novamente.”, mas se houver erro na terceira tentativa deve escrever “Arquivo inexistente. Programa terminado.”

Exercício 3.7.2

Modifique `ordenar_nomes.f90` para que ele leia 10 nomes em um arquivo e escreva uma lista alfabética em outro arquivo. O programa deve pedir ao usuário os nomes dos arquivos e verificar que o arquivo de entrada existe e que o arquivo de saída ainda não existe.

Exercício 3.7.3 (Extra)

Escreva um programa que imprima o calendário corretamente formatado do mês atual exatamente como faz o comando `cal` do Linux:

```
[heber@heber ~]$ cal
      May 2014
Su Mo Tu We Th Fr Sa
                1  2  3
 4  5  6  7  8  9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30 31
```

Há muitas fórmulas para obter o dia da semana de uma data qualquer. A que se segue foi proposta por Gauss e dá o dia da semana de uma data qualquer:

$$N = \text{mod}[d + \text{int}(2.6 * m - 0.2) + 5 \cdot \text{mod}(a, 4) + 4 \cdot \text{mod}(a, 100) + 6 \cdot \text{mod}(a, 400), 7]$$

d = dia do mês.

De março a dezembro $m = \text{mês} - 2$ e $a = \text{ano}$. Janeiro = 11 e fevereiro = 12 e $a = \text{ano} - 1$.

$N = 0$ significa domingo e assim por diante.

Esta fórmula é válida apenas para anos a partir de 1582.

Nota 3 até aqui